

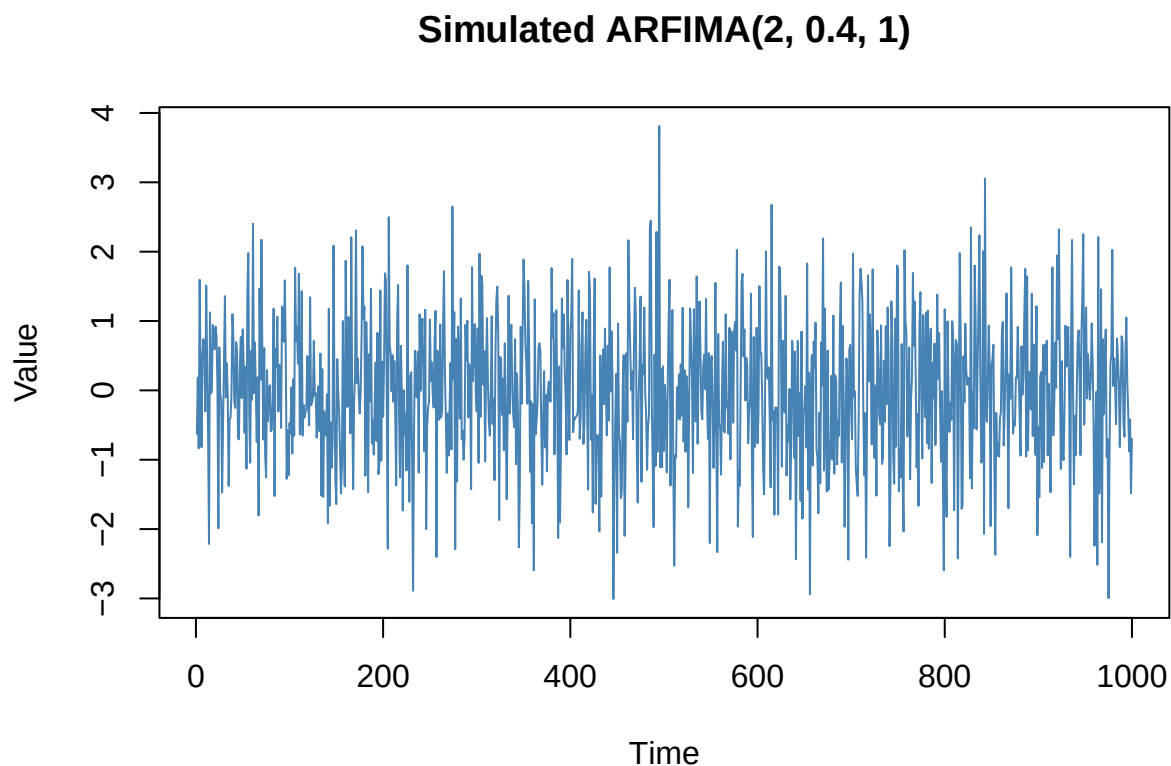
Practical 5

PART 1 - Practice 5: ARFIMA Model

1.1 Simulating the ARFIMA Model

In this section we will generate a time series using an ARFIMA(2,d,1) model. We expect “long memory” in data through the parameter d , which we for now choose as $d = 0.4$. We set a seed so that the same simulated series, and therefore the same estimates, are reproduced when the document is knitted again.

```
set.seed(1)
n <- 1000
x <- arima.sim(n, model = list(phi = c(0.2, 0.1), dfrac = 0.4, theta = 0.9))
plot(x, type = "l", main = "Simulated ARFIMA(2, 0.4, 1)", ylab = "Value", xlab = "Time", col = "steelblue")
```



1.2 Fitting the ARFIMA Model

Now we will fit the best model. To do so, we first install the package *fracdiff* which we need to estimate the best parameters. Then we use *fracdiff* with the following parameters:

- **nar=2**: specifies 2 Auto-Regressive (AR) terms
- **nma=1**: specifies 1 Moving Average (MA) term
- **drange=c(0,0.5)**: restricts the search for d to the range (0 to 0.5)

```
# install.packages("fracdiff")
library(fracdiff)

fit1<-fracdiff(x, nar=2, nma=1, drange=c(0,0.5))
```

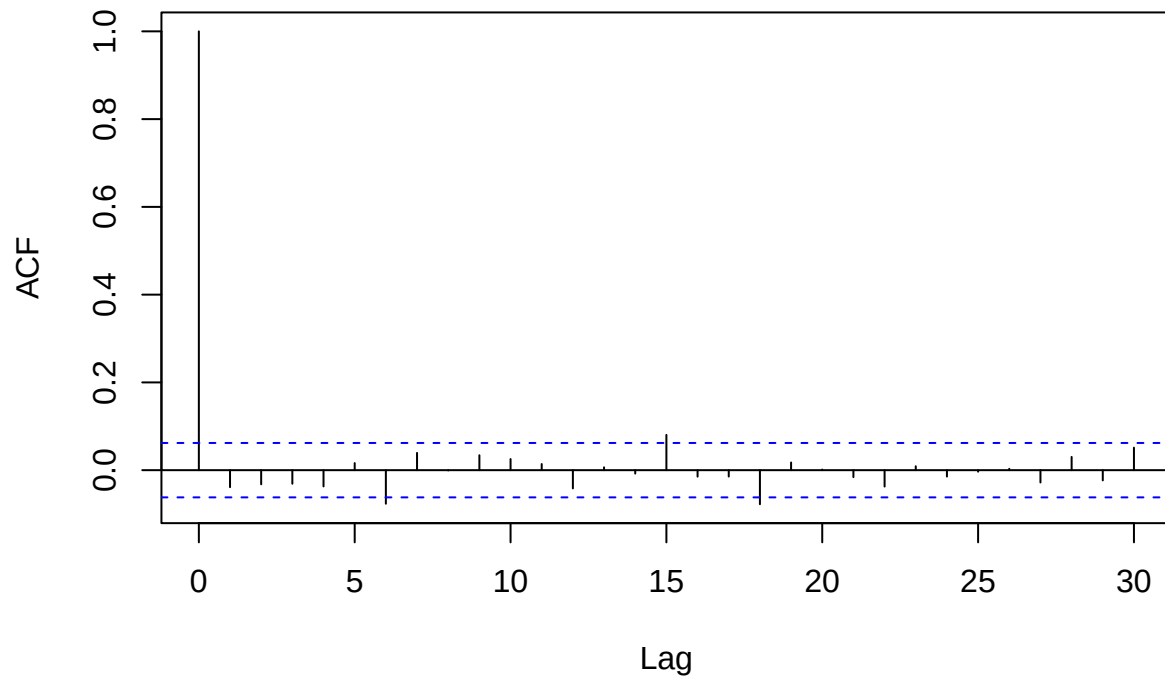
```
## Warning: unable to compute correlation matrix; maybe change 'h'
```

```
summary(fit1)
```

```
##
## Call:
##   fracdiff(x = x, nar = 2, nma = 1, drange = c(0, 0.5))
##
## *** Warning during (fdcov) fit: unable to compute correlation matrix; maybe change 'h'
##
## Coefficients:
##      Estimate
## d          0.000
## ar1        0.659
## ar2       -0.016
## ma         0.702
## sigma[eps] = 1.03307
## [d.tol = 0.0001221, M = 100, h = 1.53e-05]
## Log likelihood: -1451 ==> AIC = 2912.871 [5 deg.freedom]
```

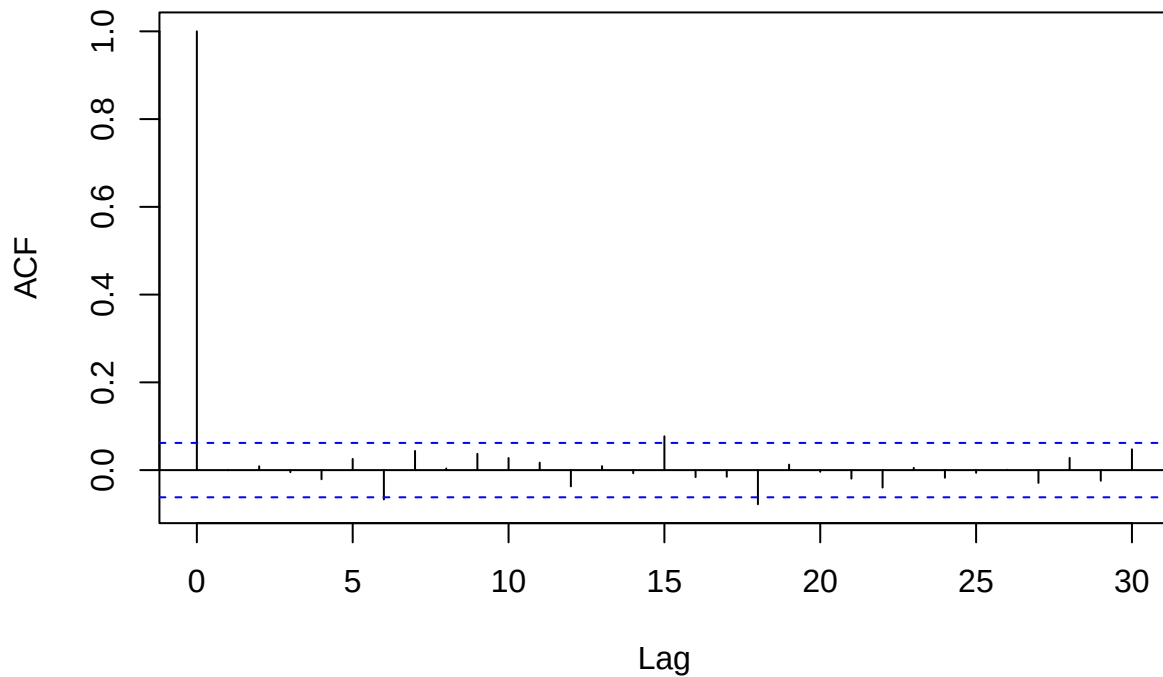
```
# Extract d and remove the long-memory component
d<-fit1$d
fit2=diffseries(x,d)
acf(fit2, main = "ACF of Fractionally Differenced Series")
```

ACF of Fractionally Differenced Series



```
# Fit the remaining ARMA structure  
fit3<-arima(fit2,order=c(2,0,1))  
fit4<-resid(fit3)  
acf(fit4, main = "ACF of Residuals (IID Noise Check)")
```

ACF of Residuals (IID Noise Check)



```
# Formal test for independence
print(Box.test(fit4, lag = floor(log(n)), type = "Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: fit4
## X-squared = 5.711, df = 6, p-value = 0.4563
```

The residual diagnostics are compatible with white noise after removing the estimated long-memory and ARMA components:

- the Ljung-Box test does not reject independence at the 5% level, so there is no strong evidence of remaining serial correlation in `fit4`
- however, the parameter recovery is poor: in this simulated example the estimated d is far from the true value 0.4, which is exactly the practical issue highlighted in the guide for more complex ARFIMA models

Observations on ACF plots:

- **ACF of Fractionally Differenced Series:** After applying `diffseries(x, d)`, the strong “long-memory” (slowly decaying) correlations should be mostly removed. Any remaining dependence should now be short-range and attributable to the ARMA part. In this run the ACF does not show a very clear residual pattern, which illustrates how difficult it can be to separate long memory from short-memory dynamics in finite samples.

- **ACF of Residuals (fit4):** Once the $ARMA(2,1)$ structure is removed from the fractionally differenced series, the ACF shows no dominant spikes outside the confidence bounds. This suggests the residuals behave approximately like white noise.

1.3 Change different values for d, phi, theta, n

Let's see how changing parameters affects the model's ability to recover d .

In particular, we analyze the influence of the sample size, the fractional differencing parameter d , and the AR/MA coefficients.

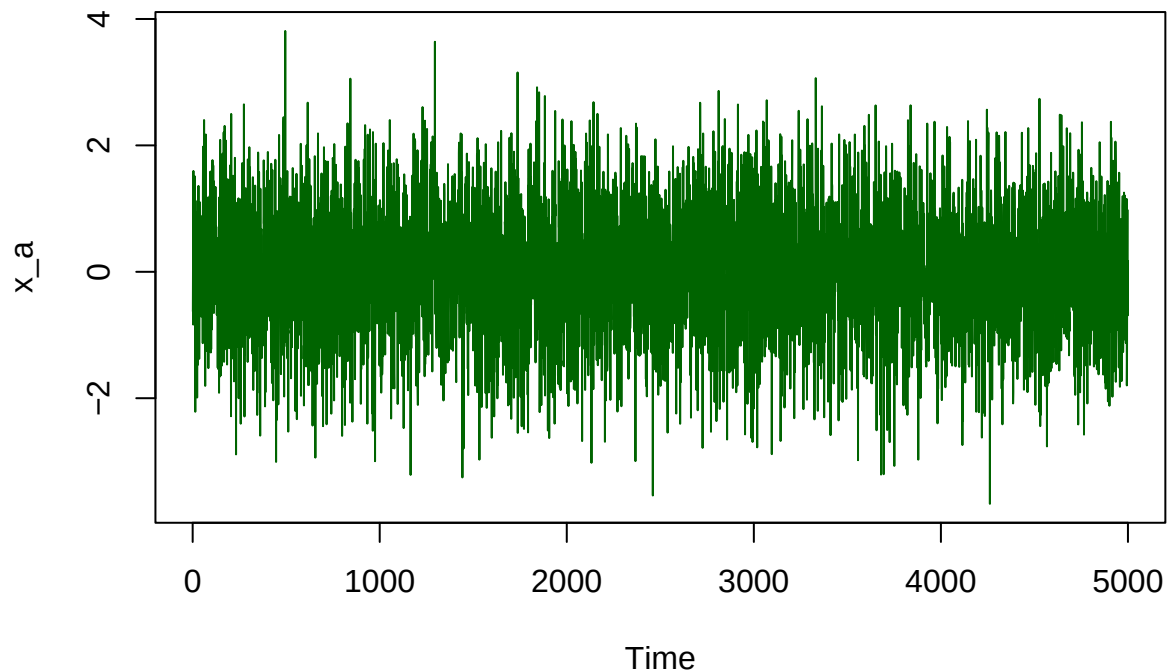
Scenario A: High Data Volume ($n = 5000$) with Stronger AR

First, we increase n to 5000 and strengthen the first AR coefficient (while weakening the MA one) to see if more data helps the model separate short-term and long-term memory.

```
set.seed(1)
n_a <- 5000
x_a <- arima.sim(n_a, model = list(phi = c(0.6, 0.1), dfrac = 0.3, theta = 0.5))

plot(x_a, type = "l", main = "ARFIMA: n=5000, d=0.3, Strong AR", col = "darkgreen")
```

ARFIMA: n=5000, d=0.3, Strong AR



```
library(fracdiff)

fit1 <- fracdiff(x_a, nar=2, nma=1, drange=c(0,0.5))
```

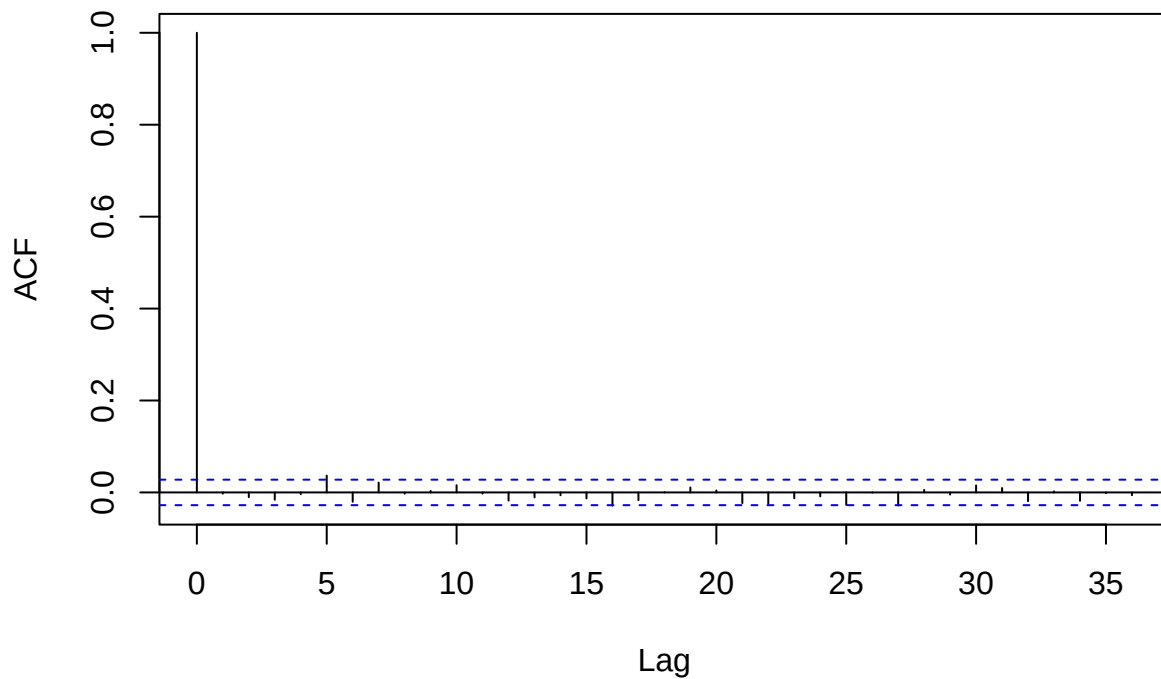
```
## Warning: unable to compute correlation matrix; maybe change 'h'
```

```
summary(fit1)
```

```
##  
## Call:  
##   fracdiff(x = x_a, nar = 2, nma = 1, drange = c(0, 0.5))  
##  
## *** Warning during (fdcov) fit: unable to compute correlation matrix; maybe change 'h'  
##  
## Coefficients:  
##      Estimate  
## d      0.009  
## ar1    0.972  
## ar2   -0.003  
## ma     0.976  
## sigma[eps] = 1.026526  
## [d.tol = 0.0001221, M = 100, h = 7.614e-05]  
## Log likelihood: -7225 ==> AIC = 14460.8 [5 deg.freedom]
```

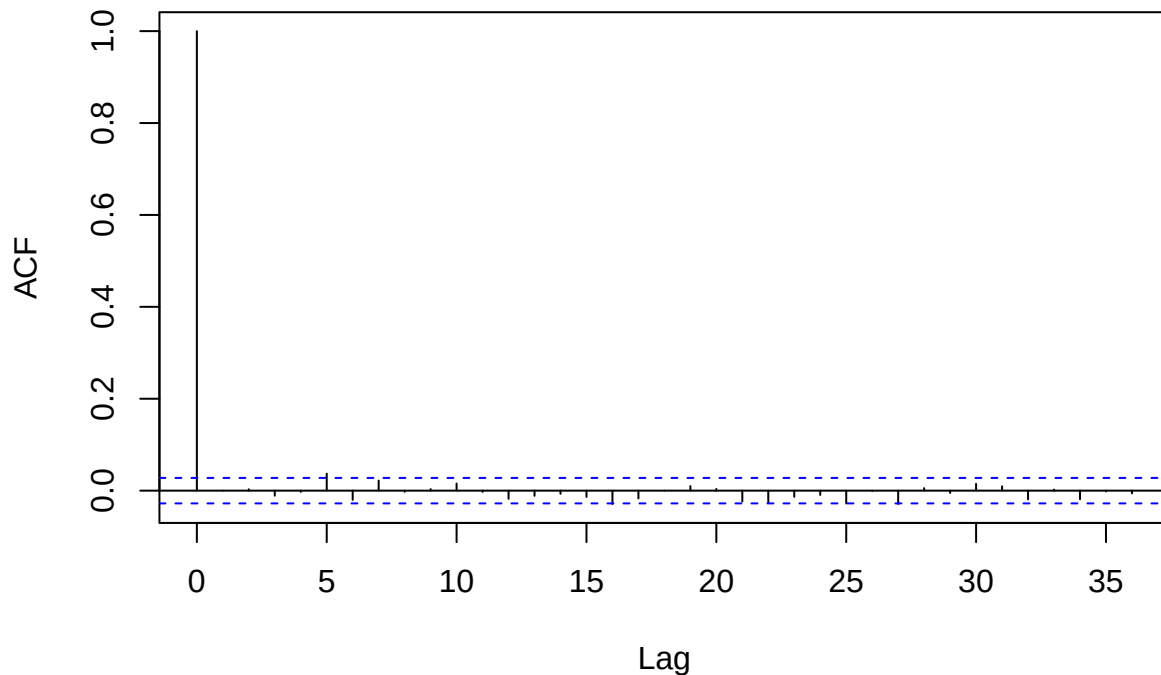
```
# Extract d and remove the long-memory component  
d<-fit1$d  
fit2=diffseries(x_a,d)  
acf(fit2, main = "ACF of Fractionally Differenced Series")
```

ACF of Fractionally Differenced Series



```
# Fit the remaining ARMA structure
fit3<-arima(fit2,order=c(2,0,1))
fit4<-resid(fit3)
acf(fit4, main = "ACF of Residuals (IID Noise Check)")
```

ACF of Residuals (IID Noise Check)



```
# Formal test for independence
print(Box.test(fit4, lag = floor(log(n_a)), type = "Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: fit4
## X-squared = 12.052, df = 8, p-value = 0.1489
```

Even with many more observations, the estimate of d is still very poor in this run: it is pushed close to 0 instead of the true $d = 0.3$. This shows that increasing n does not automatically solve the identification problem when the series contains both long memory and non-trivial ARMA dynamics.

Even though the `fracdiff` function failed to correctly identify the *source* of the memory (it missed d), the final combination of fractional differencing and $ARIMA(2, 0, 1)$ was still mathematically flexible enough to remove most of the remaining dependence, as indicated by the residual ACF and the Ljung-Box test.

Scenario B: Low Data ($n = 500$) with “Boundary” Parameters

Here we reduce n and set d close to the non-stationary boundary (0.45) and a high MA term (0.9).

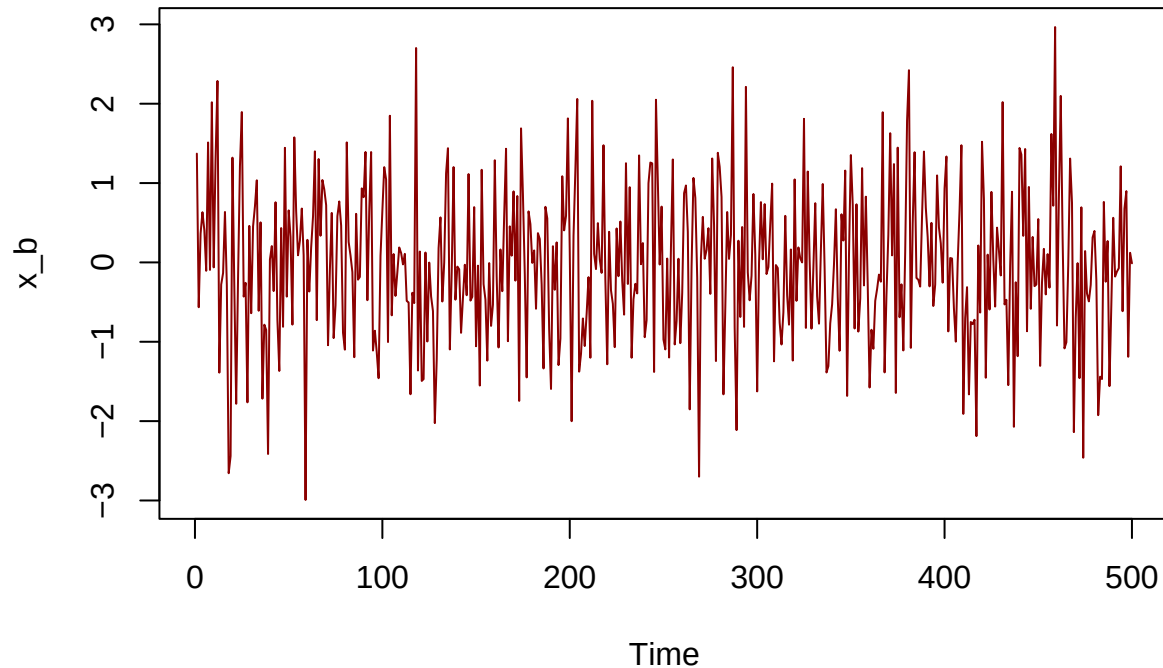
```

set.seed(42)
n_b <- 500
x_b <- arima.sim(n_b, model = list(phi = c(0.1, 0.1), dfrac = 0.45, theta = 0.9))

plot(x_b, type = "l", main = "ARFIMA: n=500, d=0.45, High MA", col = "darkred")

```

ARFIMA: n=500, d=0.45, High MA



```

library(fracdiff)

fit1<-fracdiff(x_b, nar=2, nma=1, drange=c(0,0.5))

```

```
## Warning: unable to compute correlation matrix; maybe change 'h'
```

```
summary(fit1)
```

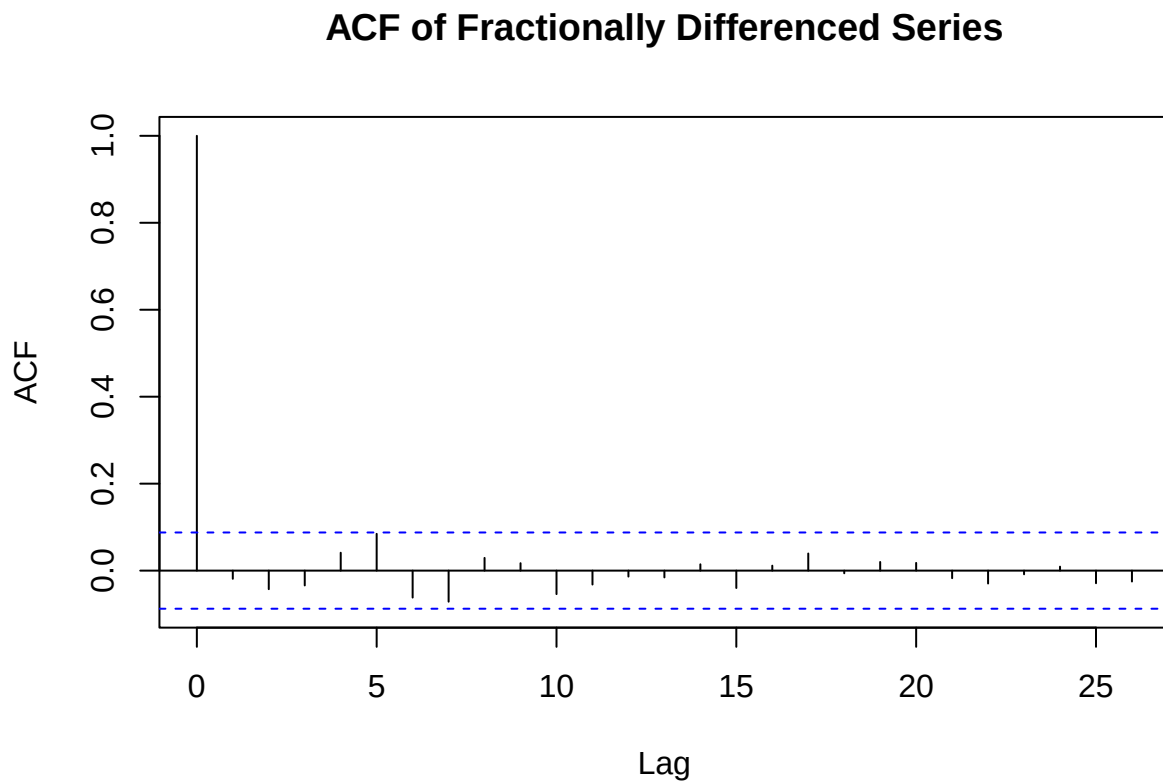
```

##
## Call:
##   fracdiff(x = x_b, nar = 2, nma = 1, drange = c(0, 0.5))
##
## *** Warning during (fdcov) fit: unable to compute correlation matrix; maybe change 'h'
##
## Coefficients:
##      Estimate
## d          0.000
## ar1       0.107

```

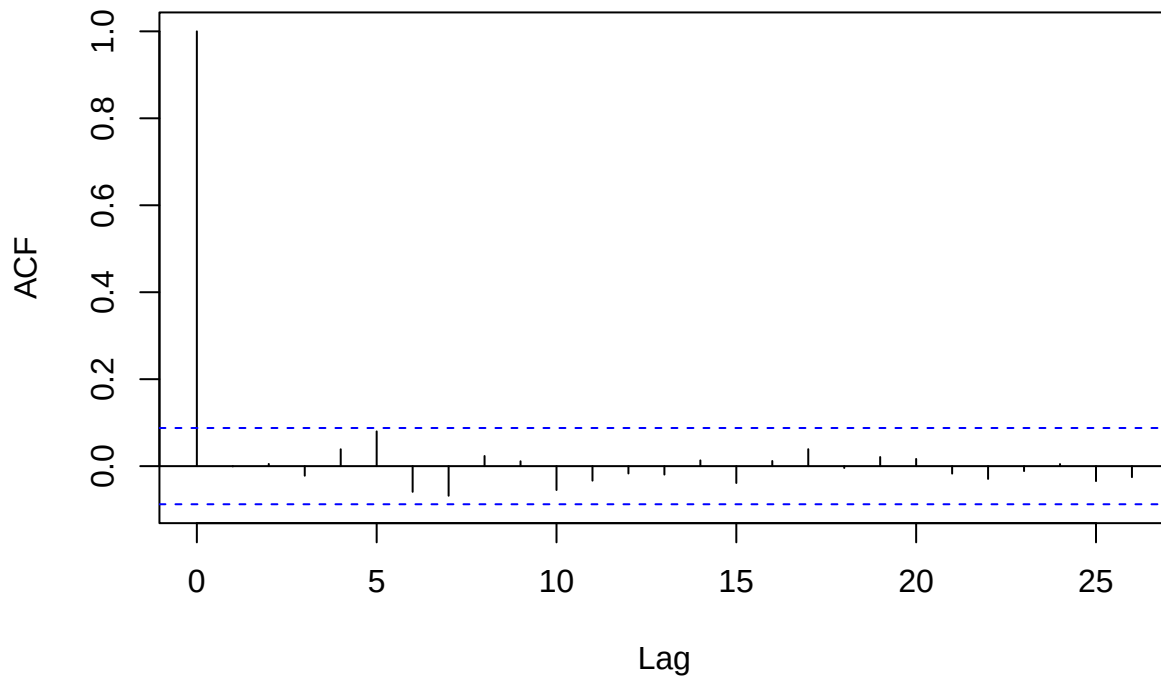
```
## ar2    -0.044
## ma      0.125
## sigma[eps] = 0.9706925
## [d.tol = 0.0001221, M = 100, h = 7.329e-06]
## Log likelihood: -694.6 ==> AIC = 1399.154 [5 deg.freedom]
```

```
# Extract d and remove the long-memory component
d<-fit1$d
fit2=diffseries(x_b,d)
acf(fit2, main = "ACF of Fractionally Differenced Series")
```



```
# Fit the remaining ARMA structure
fit3<-arima(fit2,order=c(2,0,1))
fit4<-resid(fit3)
acf(fit4, main = "ACF of Residuals (IID Noise Check)")
```

ACF of Residuals (IID Noise Check)



```
# Formal test for independence
print(Box.test(fit4, lag = floor(log(n_b)), type = "Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: fit4
## X-squared = 6.0304, df = 6, p-value = 0.4198
```

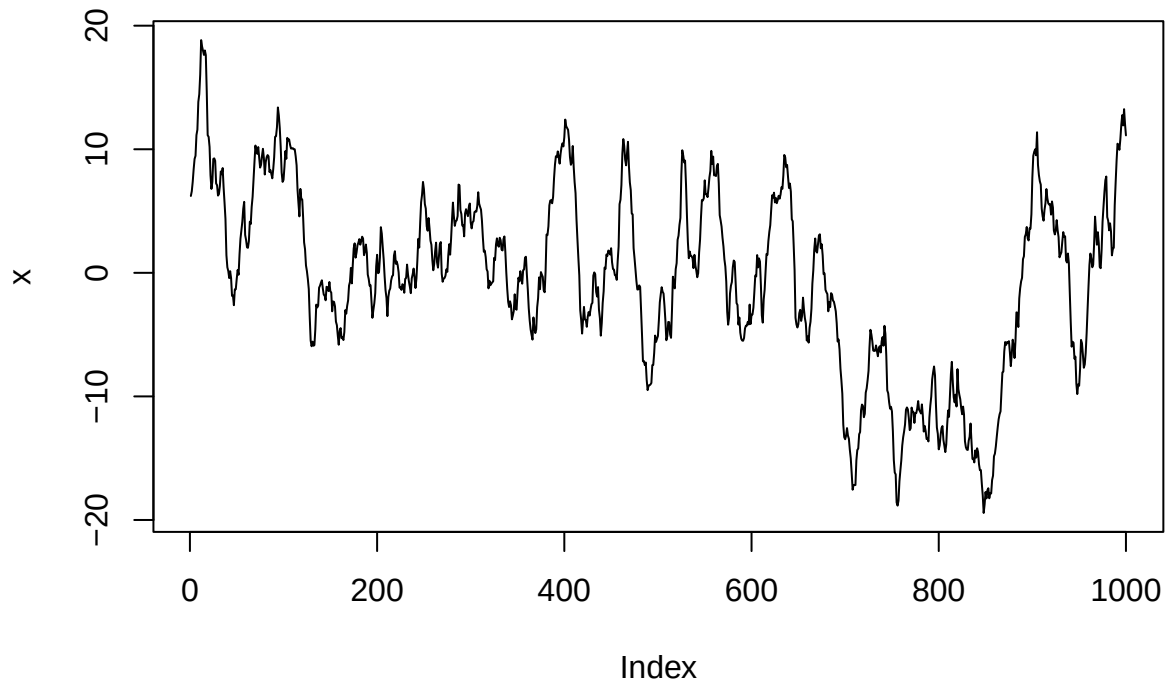
Also in this case the parameter d is not estimated correctly. This is coherent with what noted in the practical guide: adding MA components (like $\theta = 0.9$) makes things “worse” because the short-term shock of the MA term can perfectly mimic or mask the long-memory effects of d in small samples. Here the problem is amplified because $d = 0.45$ is already close to the stationarity boundary, so distinguishing persistent long memory from strong short-run dependence becomes even harder.

1.4 Simpler model: ARFIMA(1,d,0)

We now analyze a simpler ARFIMA(1,d,0) model in order to compare the estimation performance with the previous example.

```
library(fracdiff)
set.seed(42)
n<-1000
sample<-fracdiff.sim(n, ar=0.9, d=0.4)
```

```
x<-sample$series
plot(x,type="l")
```

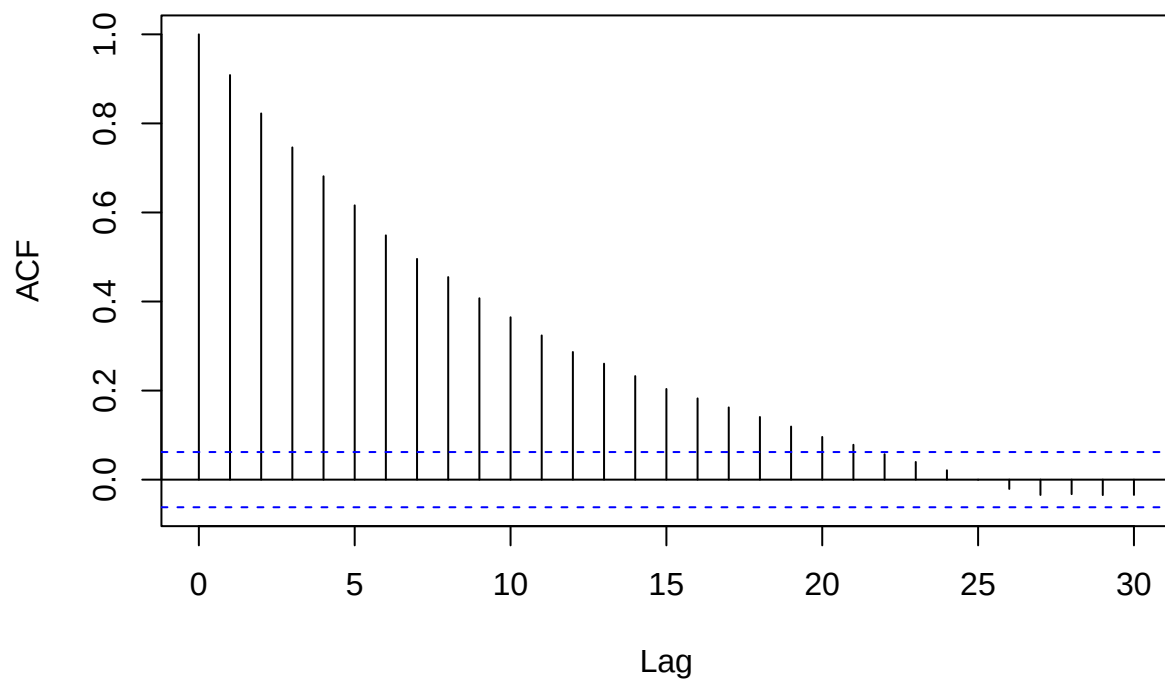


```
fit1<-fracdiff(x,nar=1)
summary(fit1)
```

```
##
## Call:
## fracdiff(x = x, nar = 1)
##
## Coefficients:
## Estimate Std. Error z value Pr(>|z|)
## d 0.36877 0.01021 36.11 <2e-16 ***
## ar 0.90602 0.01406 64.42 <2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## sigma[eps] = 1.003332
## [d.tol = 0.0001221, M = 100, h = 1.5e-05]
## Log likelihood: -1423 ==> AIC = 2851.658 [3 deg.freedom]
```

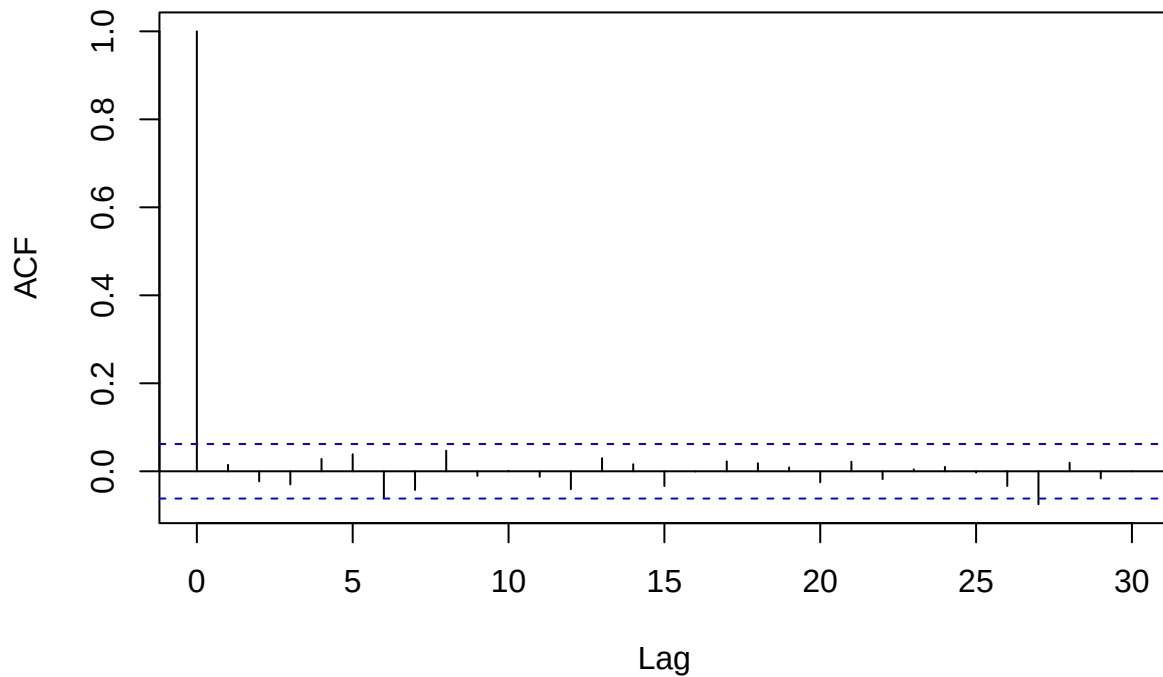
```
d<-fit1$d
y1<-diffseries(x,d)
acf(y1)
```

Series y1



```
y2<-arima(y1,order=c(1,0,0))  
z<-resid(y2)  
acf(z)
```

Series z



```
Box.test(z, lag=floor(log(n)), type=c("Ljung-Box"))
```

```
##  
## Box-Ljung test  
##  
## data: z  
## X-squared = 7.5049, df = 6, p-value = 0.2767
```

In this simpler configuration, the estimated parameters are much closer to the true simulated values. The estimated d is near 0.4 and the estimated AR coefficient is near 0.9, so the model recovers both the long-memory and short-memory parts substantially better than in the previous ARFIMA(2, d , 1) examples.

This suggests that ARFIMA estimation performs better when the underlying model structure is less complex.

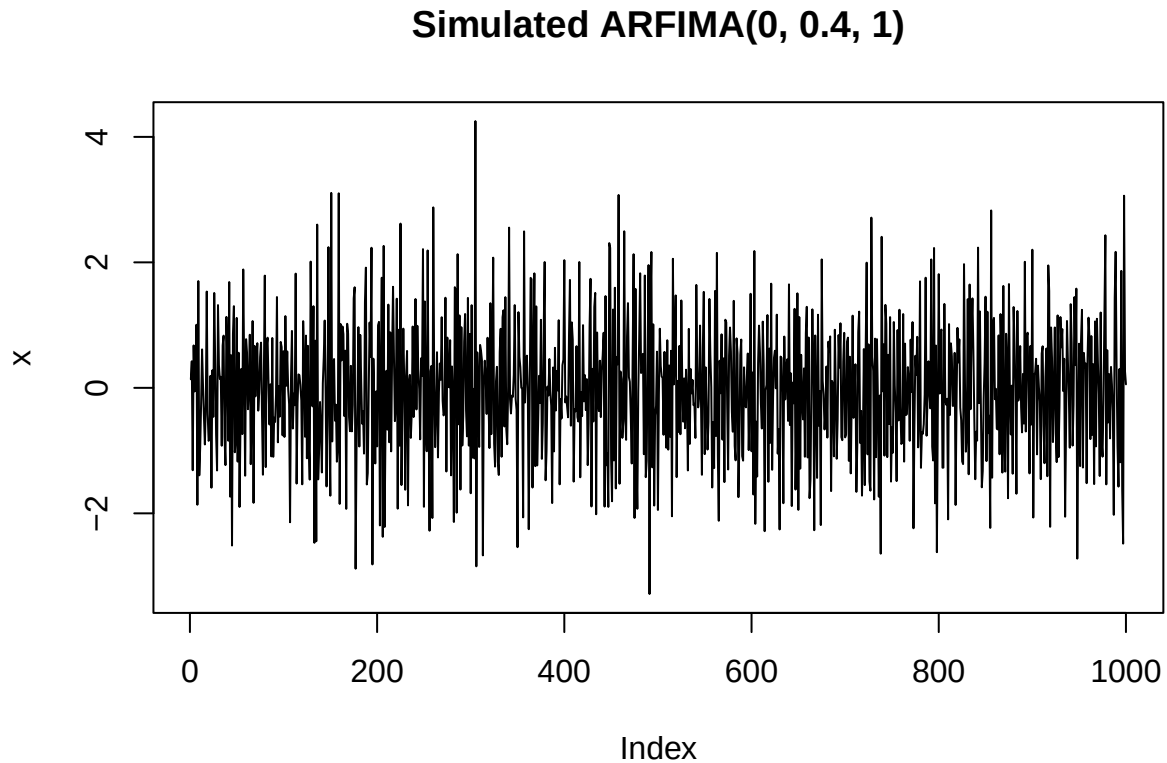
Now we will see what happens if we change to an ARFIMA(0, d , 1) model.

1.5 ARFIMA(0, d ,1) model

We keep the same idea as in the previous section, but now the short-memory component is purely moving-average. This lets us compare whether estimation remains reasonably accurate when the model has only one MA parameter together with fractional differencing.

```
library(fracdiff)  
set.seed(3)  
n <- 1000
```

```
sample <- fracdiff.sim(n, ma = 0.9, d = 0.4)
x <- sample$series
plot(x, type = "l", main = "Simulated ARFIMA(0, 0.4, 1)")
```

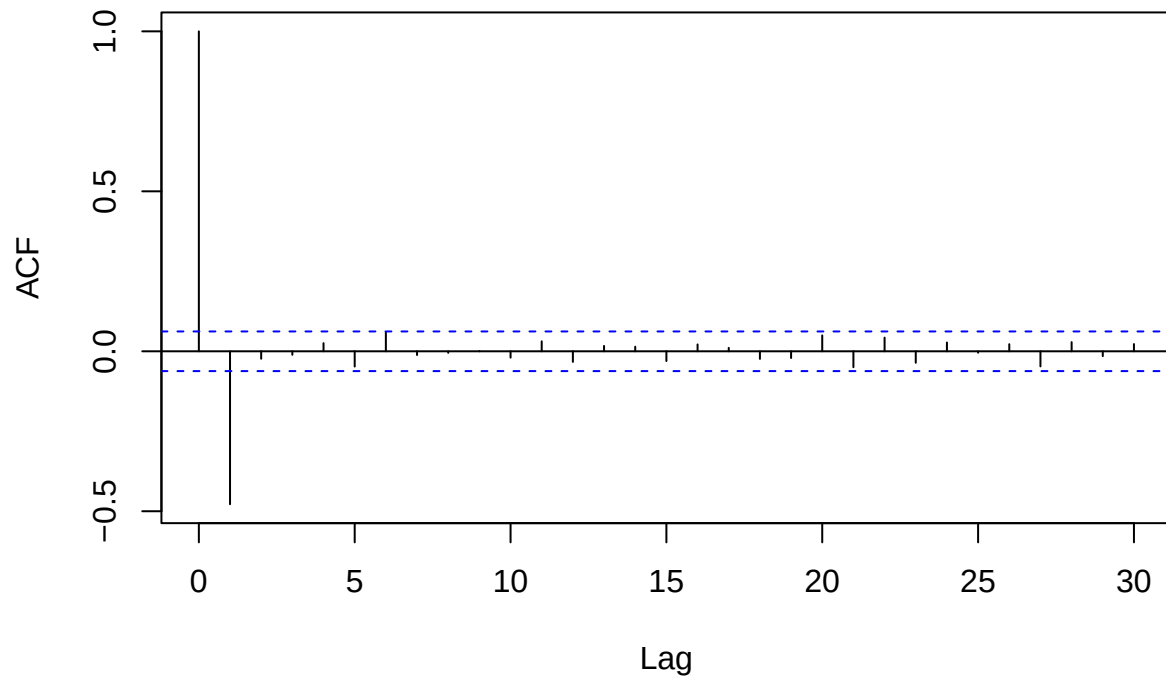


```
fit1 <- fracdiff(x, nma = 1, drange = c(0,0.5))
summary(fit1)
```

```
##
## Call:
##   fracdiff(x = x, nma = 1, drange = c(0, 0.5))
##
## Coefficients:
##   Estimate Std. Error z value Pr(>|z|)
## d  0.410063  0.009437   43.45  <2e-16 ***
## ma 0.920303  0.012245   75.16  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## sigma[eps] = 0.9982342
## [d.tol = 0.0001221, M = 100, h = 1.495e-05]
## Log likelihood: -1418 ==> AIC = 2841.914 [3 deg.freedom]
```

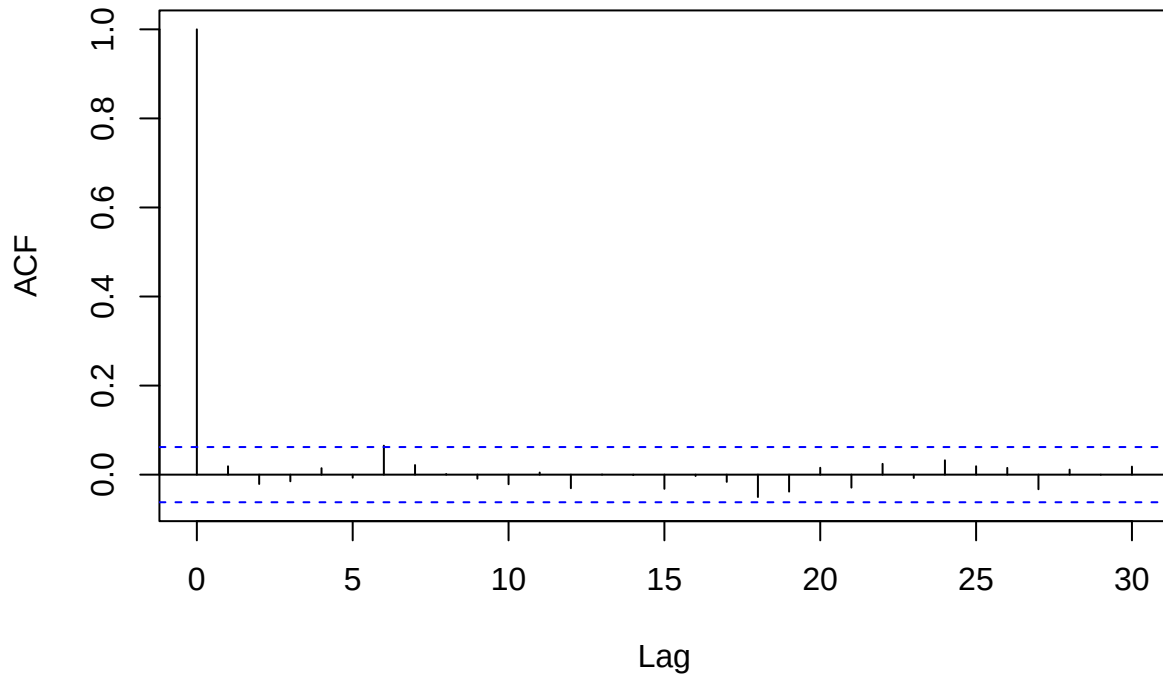
```
d <- fit1$d
y1 <- diffseries(x, d)
acf(y1, main = "ACF after Fractional Differencing")
```

ACF after Fractional Differencing



```
y2 <- arima(y1, order = c(0,0,1))  
z <- resid(y2)  
acf(z, main = "ACF of Residuals")
```

ACF of Residuals



```
Box.test(z, lag = floor(log(n)), type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: z  
## X-squared = 5.5399, df = 6, p-value = 0.4766
```

In this case the estimation is again clearly better than in the ARFIMA(2, d , 1) examples. The estimated d is close to the true value 0.4, and the estimated MA coefficient is also close to the simulated $\theta = 0.9$.

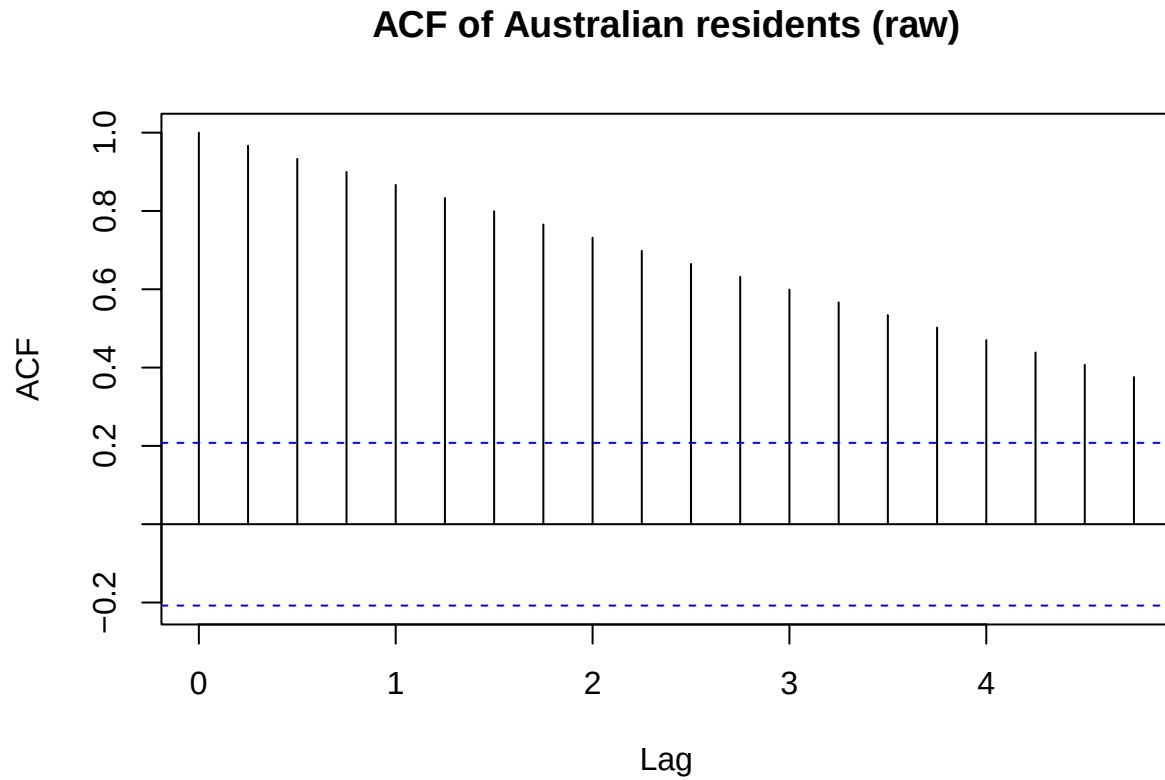
After removing the fractional dependence, the ACF of y_1 still shows the short-run pattern expected from an MA component, and after fitting the MA(1) model the residual ACF no longer shows important autocorrelation. The Ljung-Box test is also non-significant, so the residuals are compatible with white noise.

Together with Section 1.4, this confirms the main message of the practical: ARFIMA estimation tends to work better for simpler model structures, whereas richer ARMA dynamics can hide the true fractional parameter d .

1.6 Real data: Australian residents

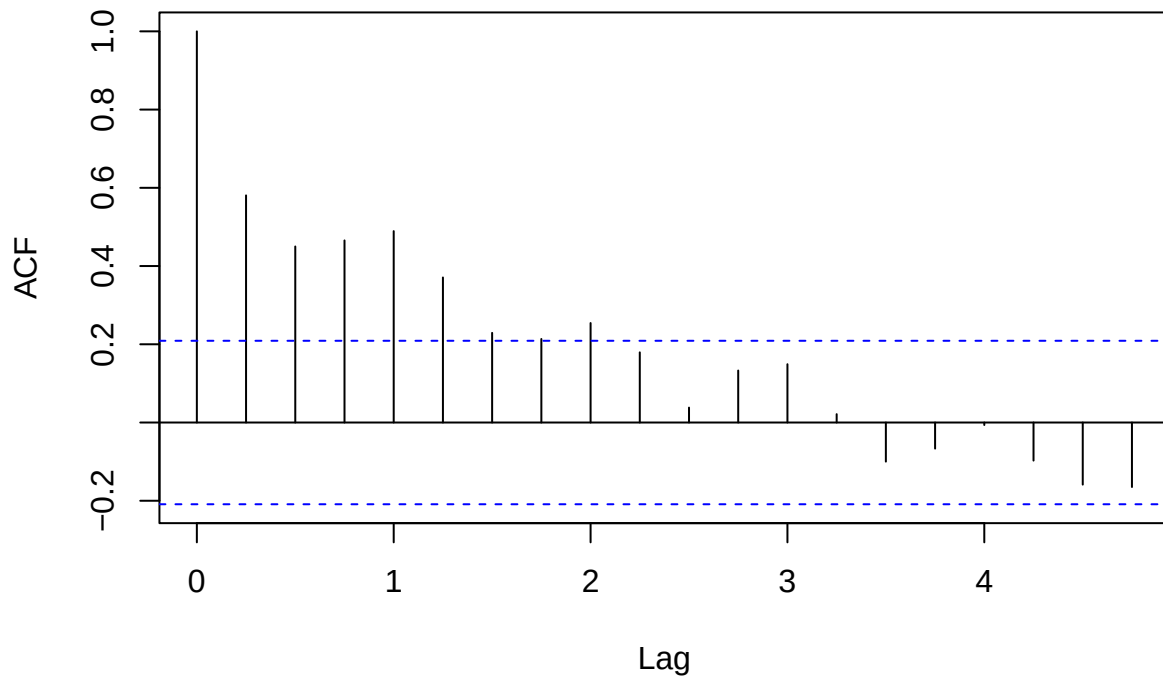
We now apply ARFIMA modeling techniques to a real dataset. The `austres` dataset contains quarterly estimates of the Australian resident population between 1972 and 1993.

```
library("datasets")
x<-austres
acf(x, main = "ACF of Australian residents (raw)")
```



```
# First difference to handle trend
dx<-diff(x)
acf(dx, main = "ACF of differenced Australian residents")
```

ACF of differenced Australian residents



```
# Fit fractional d to the differenced series
fdx<-fracdiff(dx,nar=0,nma=0, drange=c(0,0.5))
summary(fdx)
```

```
##
## Call:
## fracdiff(x = dx, nar = 0, nma = 0, drange = c(0, 0.5))
##
## Coefficients:
## Estimate Std. Error z value Pr(>|z|)
## d 4.171e-01 3.457e-06 120655 <2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## sigma[eps] = 9.962333
## [d.tol = 0.0001221, M = 100, h = 3.467e-06]
## Log likelihood: -328 ==> AIC = 660.0368 [2 deg.freedom]
```

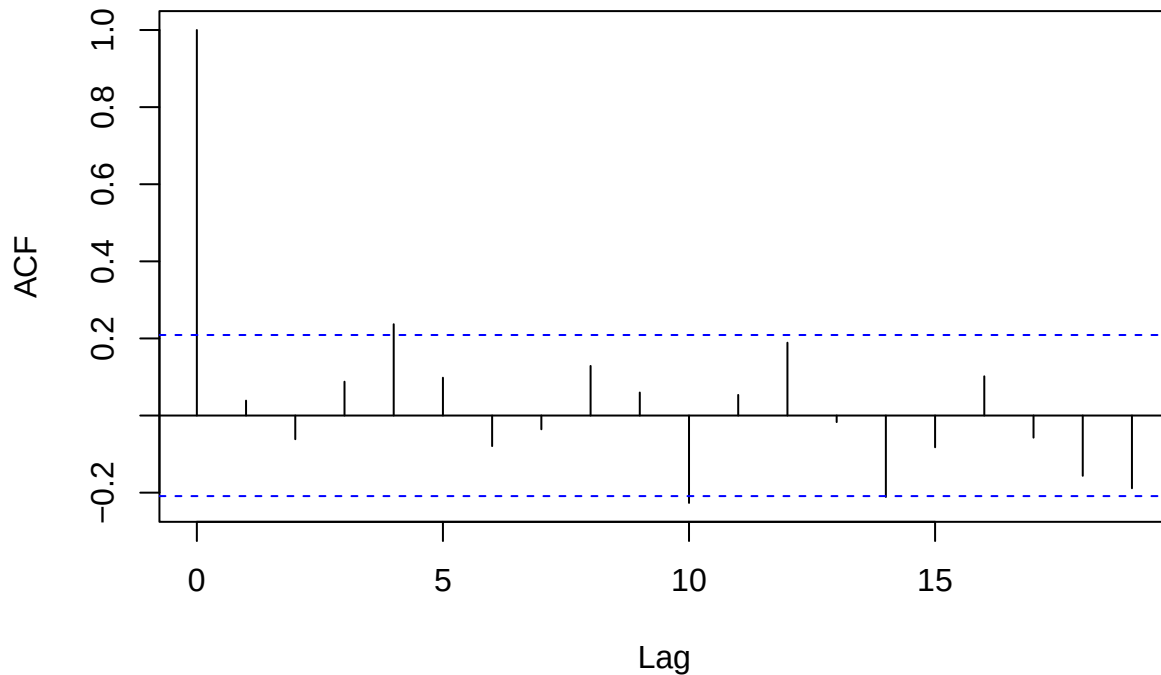
```
d=fdx$d
d
```

```
## [1] 0.4170919
```

```
# Remove fractional noise
resid<-diffseries(dx,d)
```

```
y<-resid-mean(resid)
acf(y, main = "Final ACF: white noise check")
```

Final ACF: white noise check



```
Box.test(y,lag=floor(log(length(x))),type=c("Ljung-Box"))
```

```
##
## Box-Ljung test
##
## data: y
## X-squared = 6.4922, df = 4, p-value = 0.1653
```

Observations:

- **ACF of Raw Data:** shows extremely slow linear decay, indicating the series is **non-stationary** and has a strong trend.
- **ACF of Differenced Data:** after `diff(x)`, the trend is removed, but significant autocorrelations remain at seasonal lags, which shows that the series still has serial dependence.
- **Final ACF:** after removing the estimated fractional d component, the remaining autocorrelations are much smaller, but the plot can still show some moderate spikes, especially around seasonal lags. So the result is closer to white noise than the raw series, but not perfectly featureless.

The **Ljung-Box test** on y does not reject independence at the 5% level, so this transformation removes a large part of the dependence. However, the estimated fractional parameter is not tiny, and the final ACF suggests that some structure may still remain. A cautious conclusion is therefore that first differencing plus fractional differencing gives a much more stationary series, but not necessarily a perfect white noise process.

Overall, these results show that ARFIMA models are useful for capturing long-memory behavior, but their estimation is clearly more reliable in simpler specifications than in more complex ARMA settings.

PART 2 - Practice 5: GARCH Models

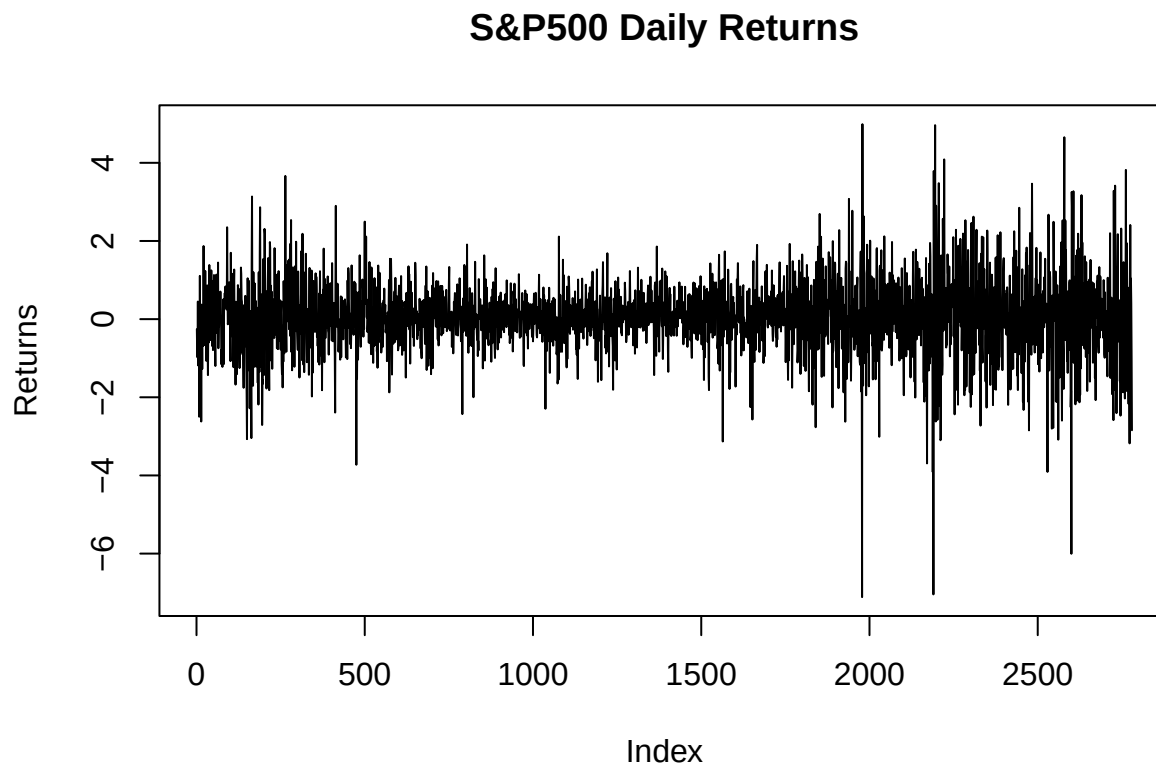
1.1 White but not IID noise residuals

We analyze the daily returns of the S&P500 and discover that: - the returns themselves behave like white noise - but their variance changes over time - squared returns show strong autocorrelation, indicating volatility clustering

This motivates ARCH/GARCH models.

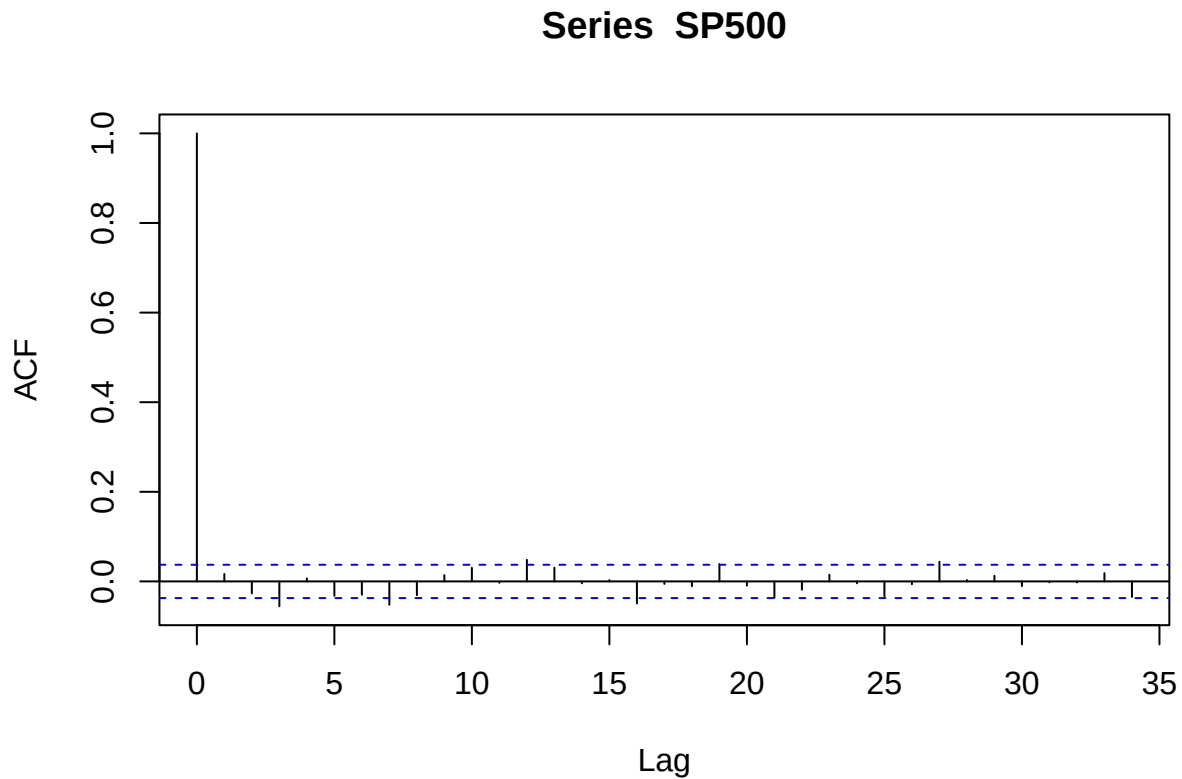
```
library(MASS)
data(SP500)

plot(SP500, type = "l", main = "S&P500 Daily Returns", ylab = "Returns")
```



Explanation: - SP500 contains daily stock returns - The plot shows returns fluctuating around zero - We observe periods of low and high volatility, which is a hallmark of financial data This already suggests heteroskedasticity (non-constant variance).

```
acf(SP500)
```



```
Box.test(SP500, type = "Ljung-Box")
```

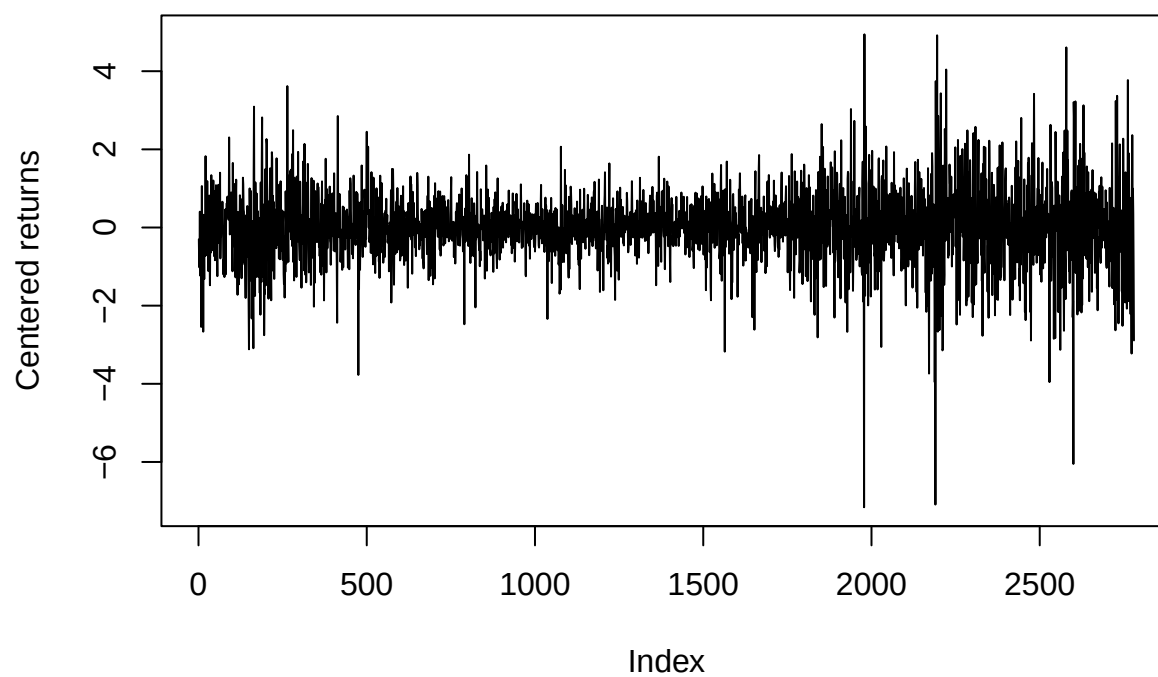
```
##  
## Box-Ljung test  
##  
## data: SP500  
## X-squared = 0.76379, df = 1, p-value = 0.3821
```

Explanation: - The ACF (Autocorrelation Function) measures linear dependence.

The Ljung-Box test is usually not significant for the raw returns, so the series is close to white noise in the mean. However, this does not mean that the conditional variance is constant. To see that, we remove the sample mean and look at the squared centered returns.

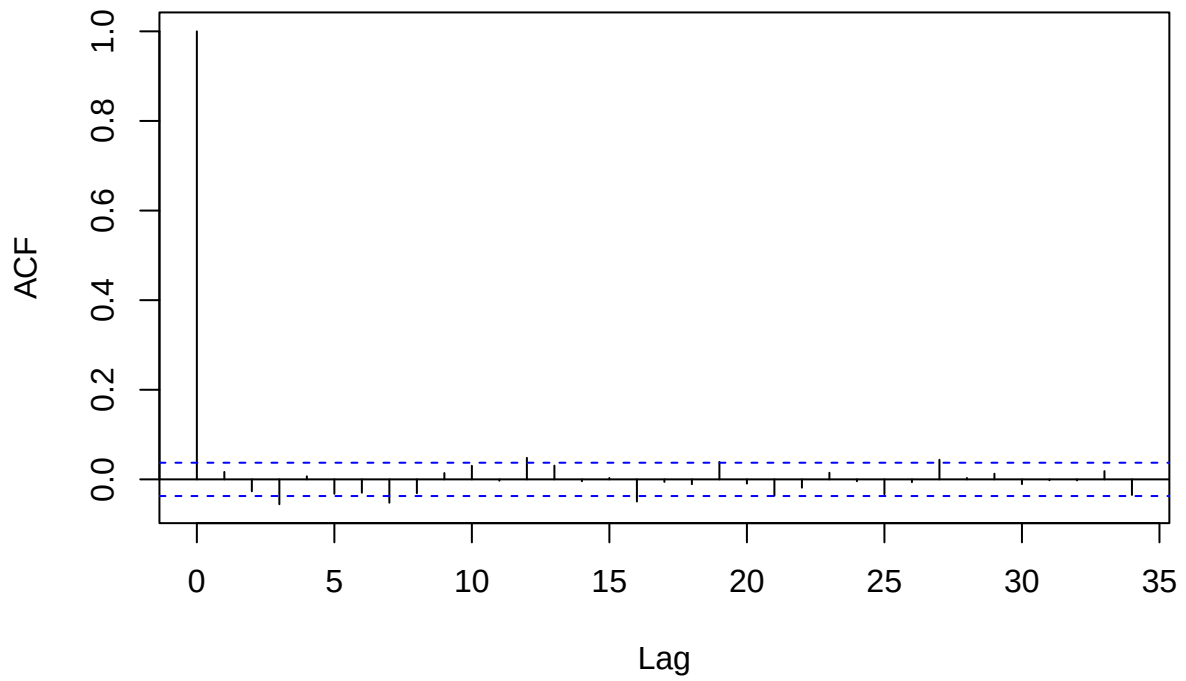
```
m <- mean(SP500)  
x <- SP500 - m  
  
plot(x, type = "l", main = "Centered S&P500 Returns", ylab = "Centered returns")
```

Centered S&P500 Returns



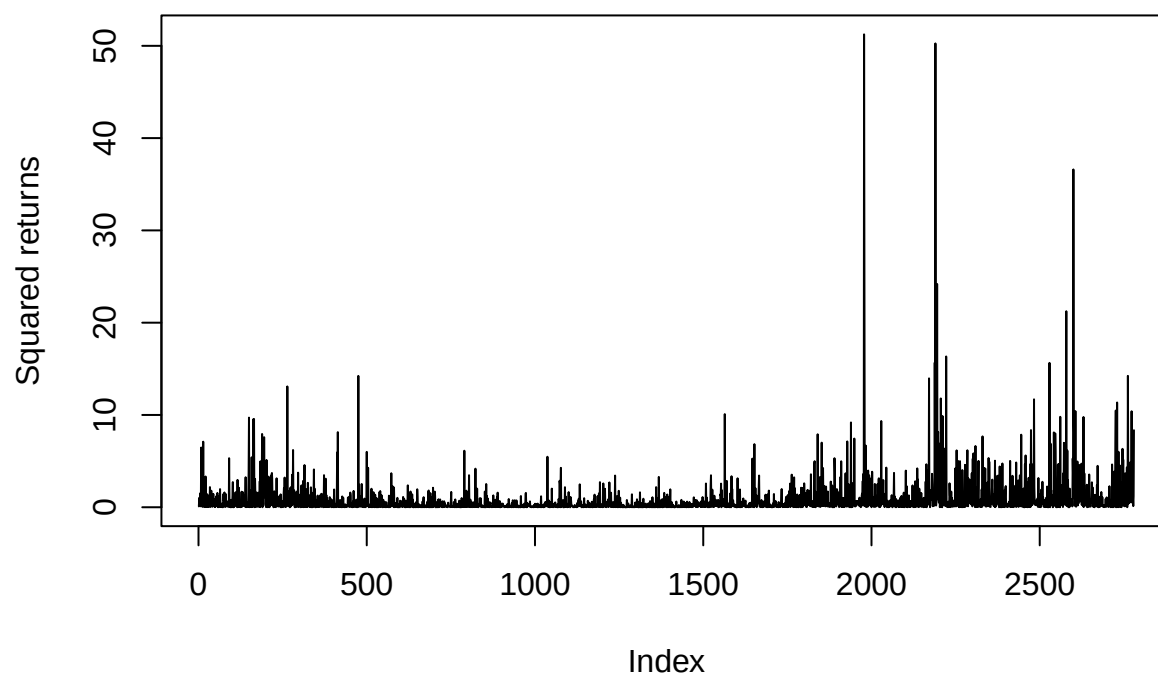
```
acf(x, main = "ACF of Centered Returns")
```

ACF of Centered Returns



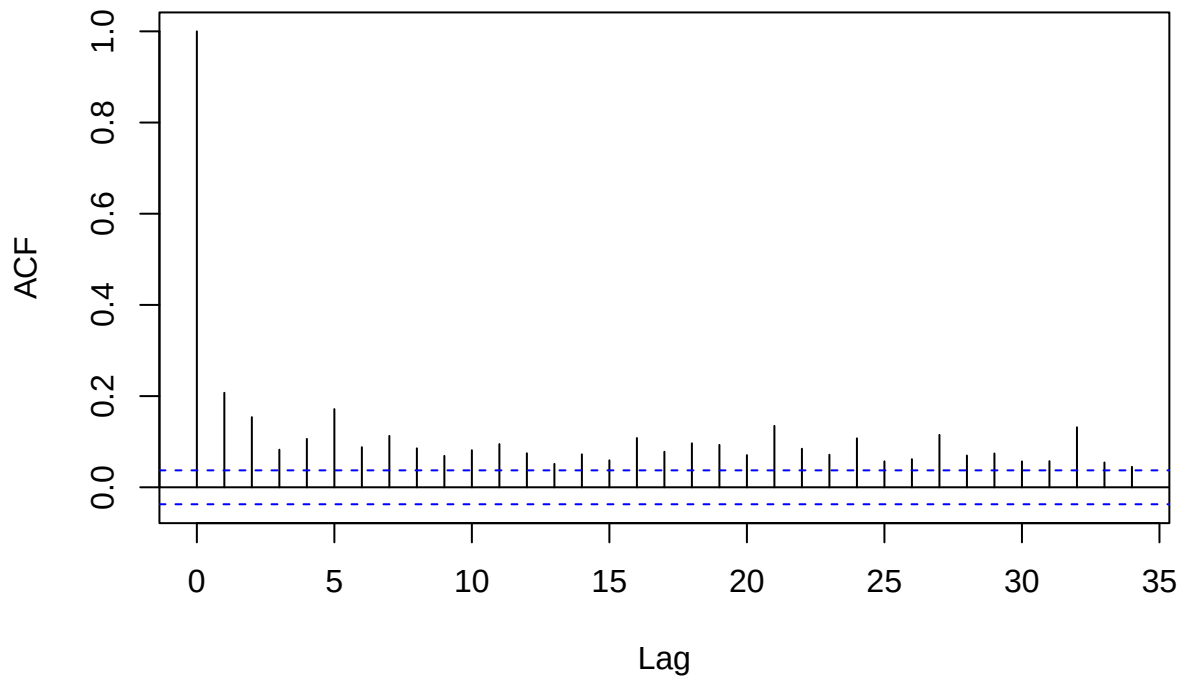
```
y <- x^2  
plot(y, type = "l", main = "Squared Centered S&P500 Returns", ylab = "Squared returns")
```

Squared Centered S&P500 Returns



```
acf(y, main = "ACF of Squared Returns")
```

ACF of Squared Returns



```
Box.test(y, type = "Ljung-Box")
```

```
##  
## Box-Ljung test  
##  
## data: y  
## X-squared = 119.52, df = 1, p-value < 2.2e-16
```

The squared series is no longer white noise. Its autocorrelation is typically much stronger than that of the returns themselves, which means that the volatility is clustered over time. This is exactly the pattern that ARCH and GARCH models are designed to describe.

1.2 Generating ARCH and GARCH models

We now simulate artificial ARCH and GARCH processes to see how they produce time-varying variance and dependence in the squared values.

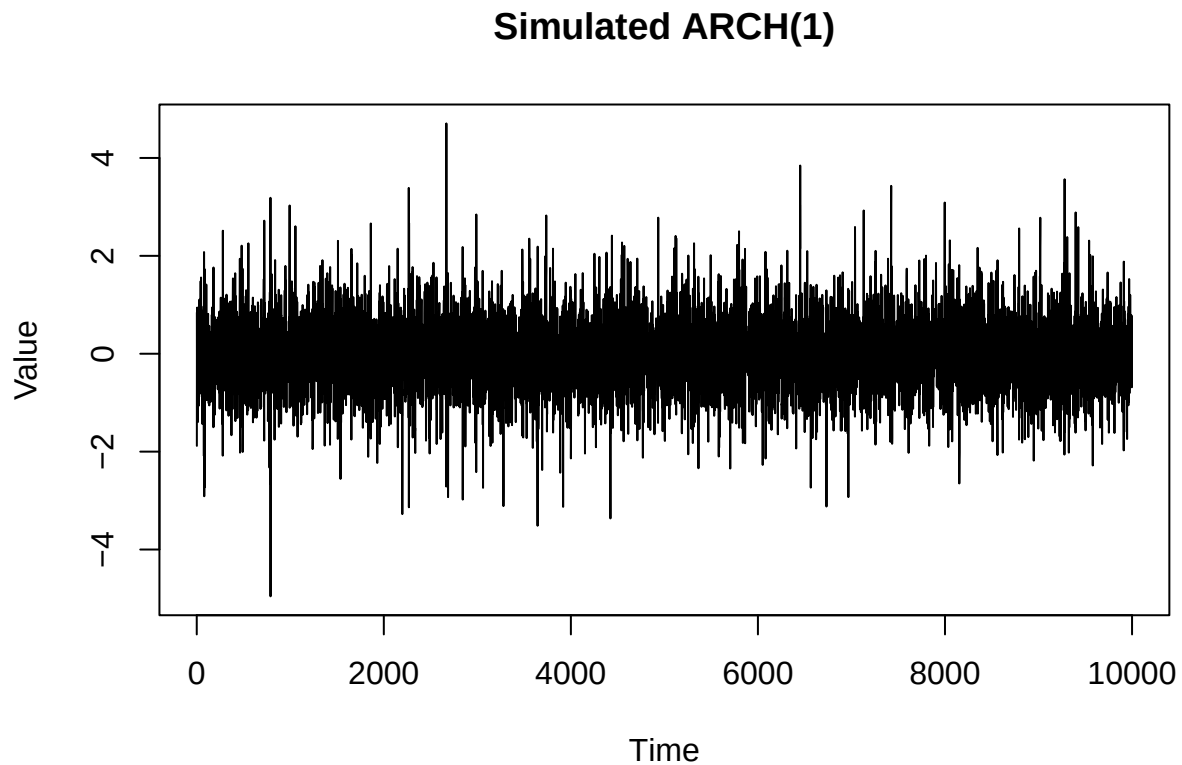
ARCH(1) simulation

```
a0 <- 0.25  
a1 <- 0.5  
x <- numeric(9999)
```

```
x0 <- rnorm(1)

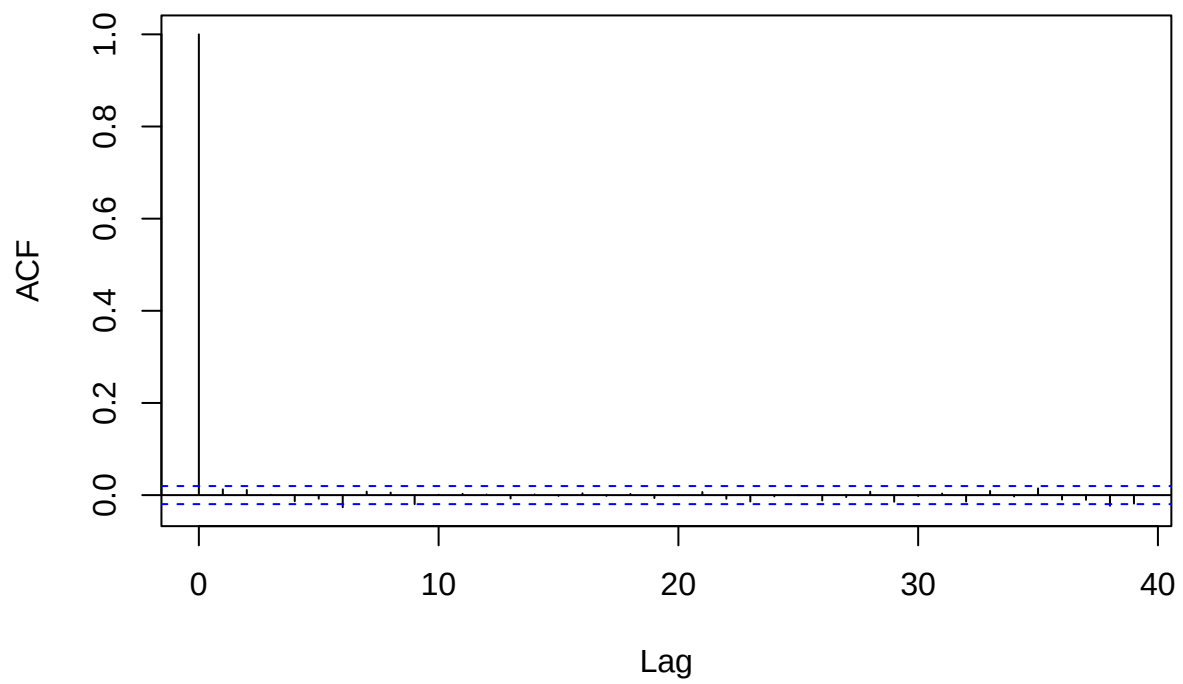
for (i in 1:9999) {
  x0 <- rnorm(1, sd = sqrt(a0 + a1 * x0^2))
  x[i] <- x0
}

plot(x, type = "l", main = "Simulated ARCH(1)", ylab = "Value", xlab = "Time")
```



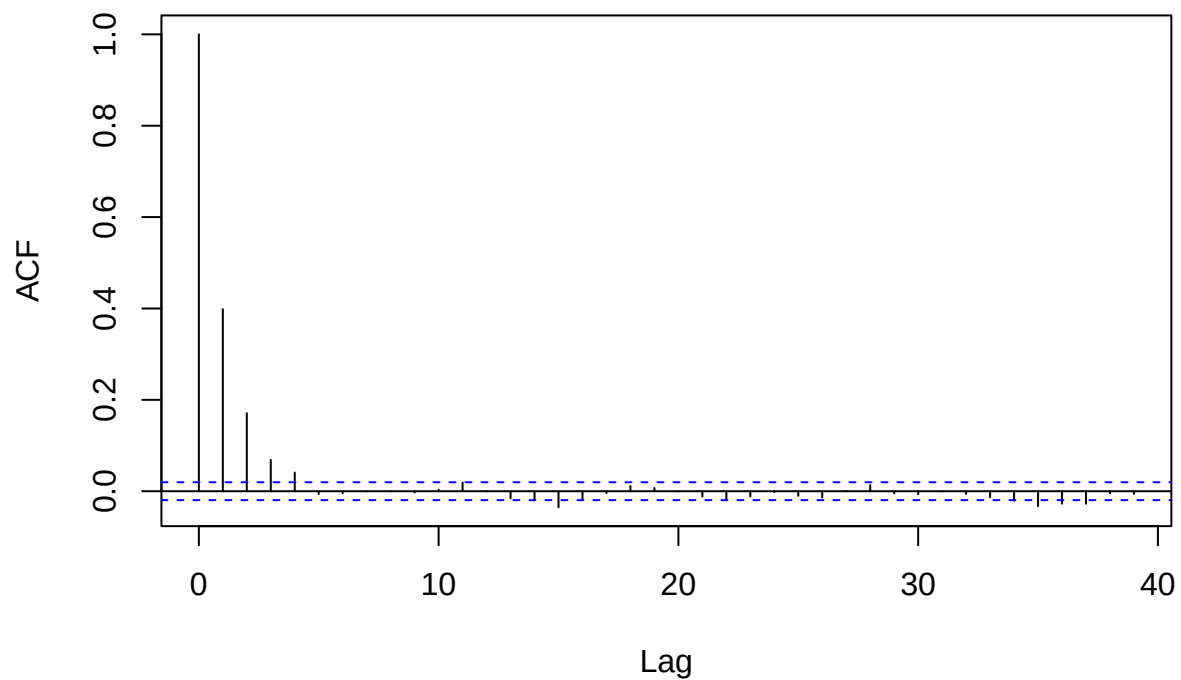
```
acf(x, main = "ACF of ARCH(1) series")
```

ACF of ARCH(1) series



```
acf(x^2, main = "ACF of squared ARCH(1) series")
```

ACF of squared ARCH(1) series



The ARCH(1) process does not usually show strong autocorrelation in the raw series, but the squared series exhibits dependence. That is the signature of changing conditional variance.

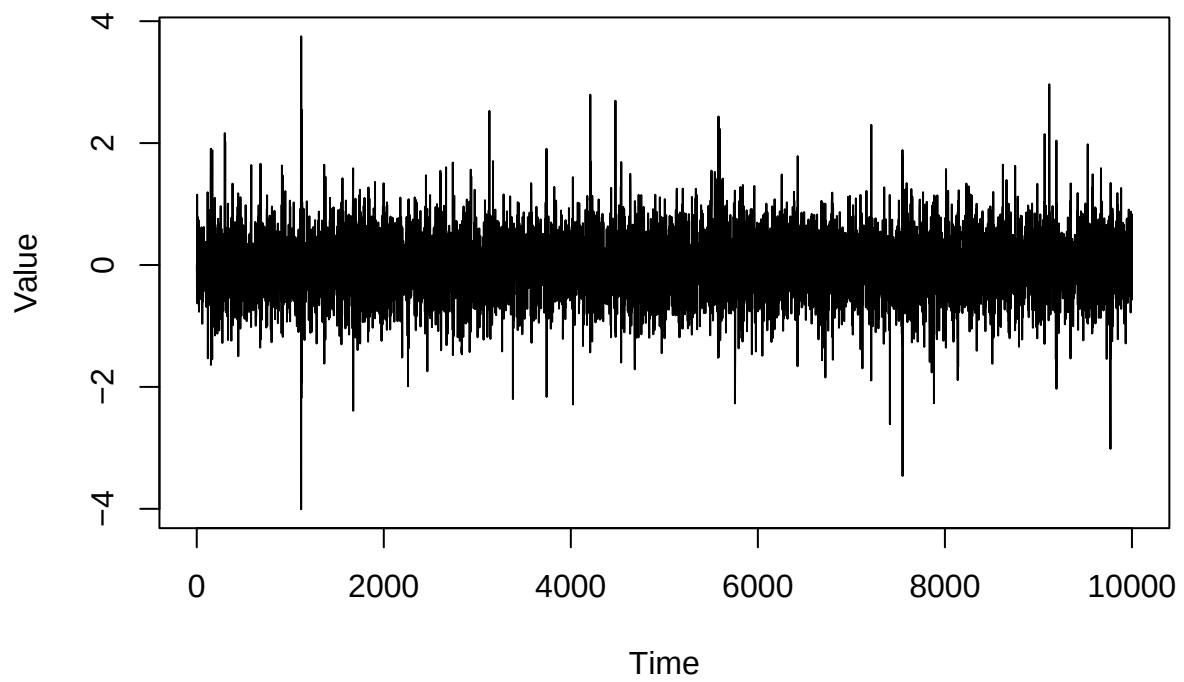
GARCH(1,1) simulation

```
n <- 10000
a0 <- 0.1
a1 <- 0.4
b1 <- 0.2
w <- rnorm(n)
y <- rep(0, n)
h <- rep(0, n)

for (i in 2:n) {
  h[i] <- a0 + a1 * (y[i - 1]^2) + b1 * h[i - 1]
  y[i] <- w[i] * sqrt(h[i])
}

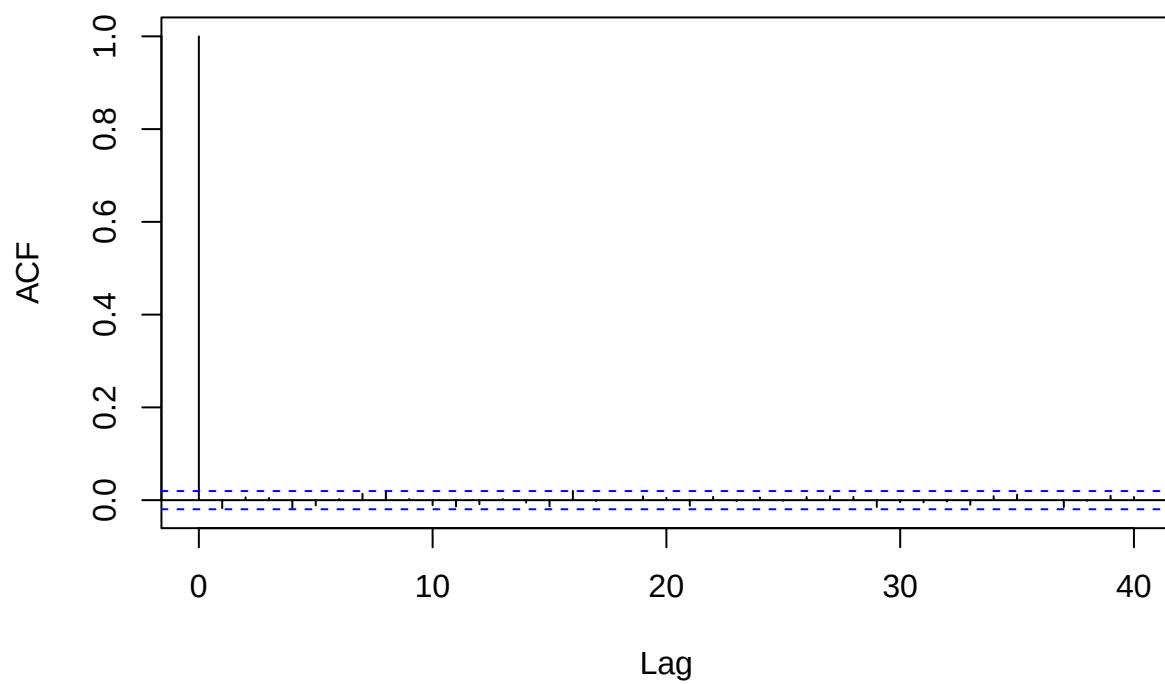
plot(y, type = "l", main = "Simulated GARCH(1,1)", ylab = "Value", xlab = "Time")
```

Simulated GARCH(1,1)



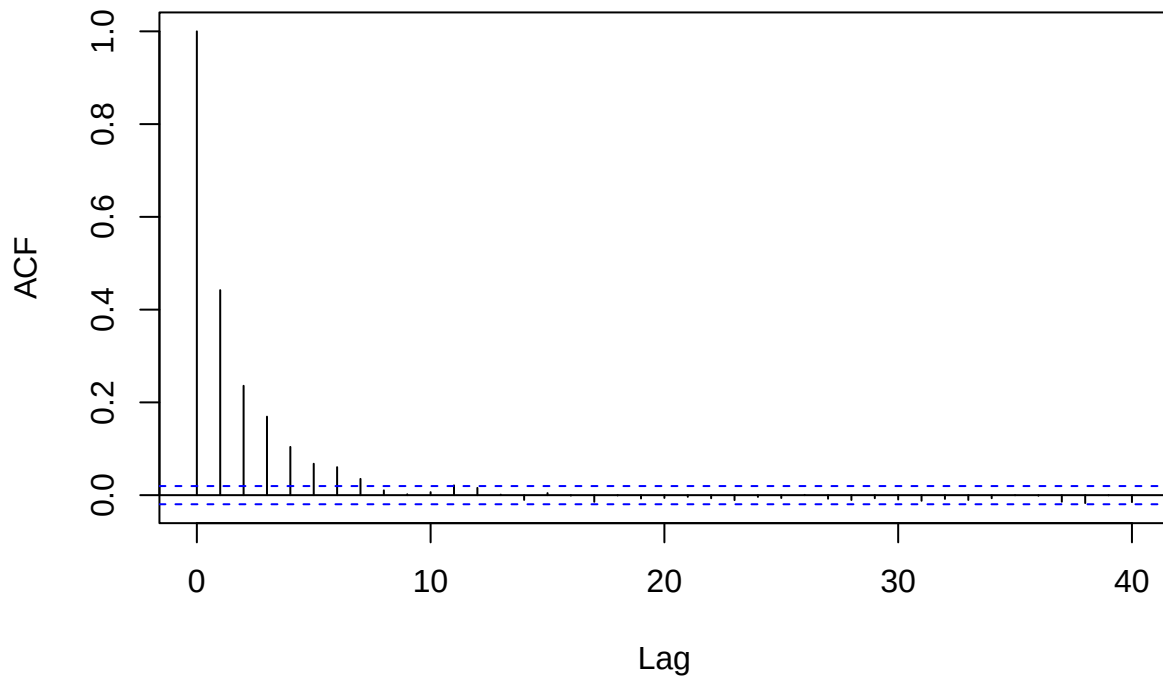
```
acf(y, main = "ACF of GARCH(1,1) series")
```

ACF of GARCH(1,1) series



```
acf(y2, main = "ACF of squared GARCH(1,1) series")
```

ACF of squared GARCH(1,1) series



Compared with ARCH, the GARCH model includes past conditional variance through the b_1 parameter. This produces smoother volatility clusters and a slower decay in the autocorrelation of squared returns.

1.3 ARCH(2) simulation

We now repeat the simulation for an ARCH(2) model.

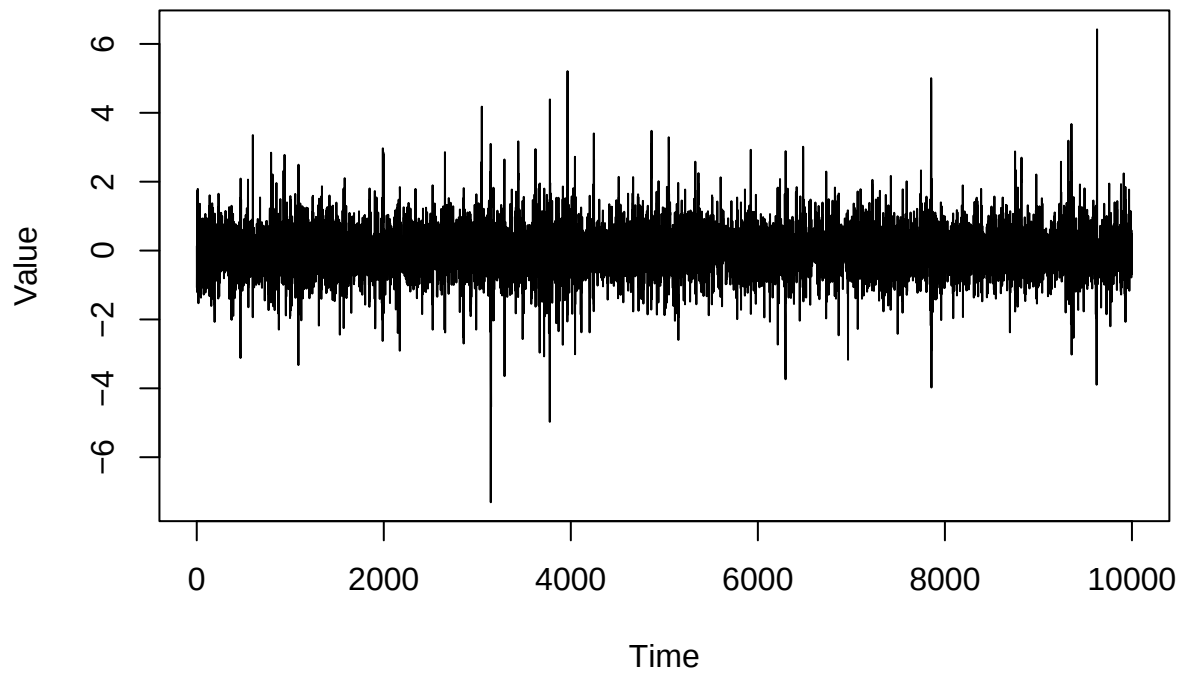
```
a0 <- 0.2
a1 <- 0.4
a2 <- 0.2
n <- 10000
x <- numeric(n)

x[1:2] <- rnorm(2)

for (i in 3:n) {
  h_i <- a0 + a1 * x[i - 1]^2 + a2 * x[i - 2]^2
  x[i] <- rnorm(1, sd = sqrt(h_i))
}

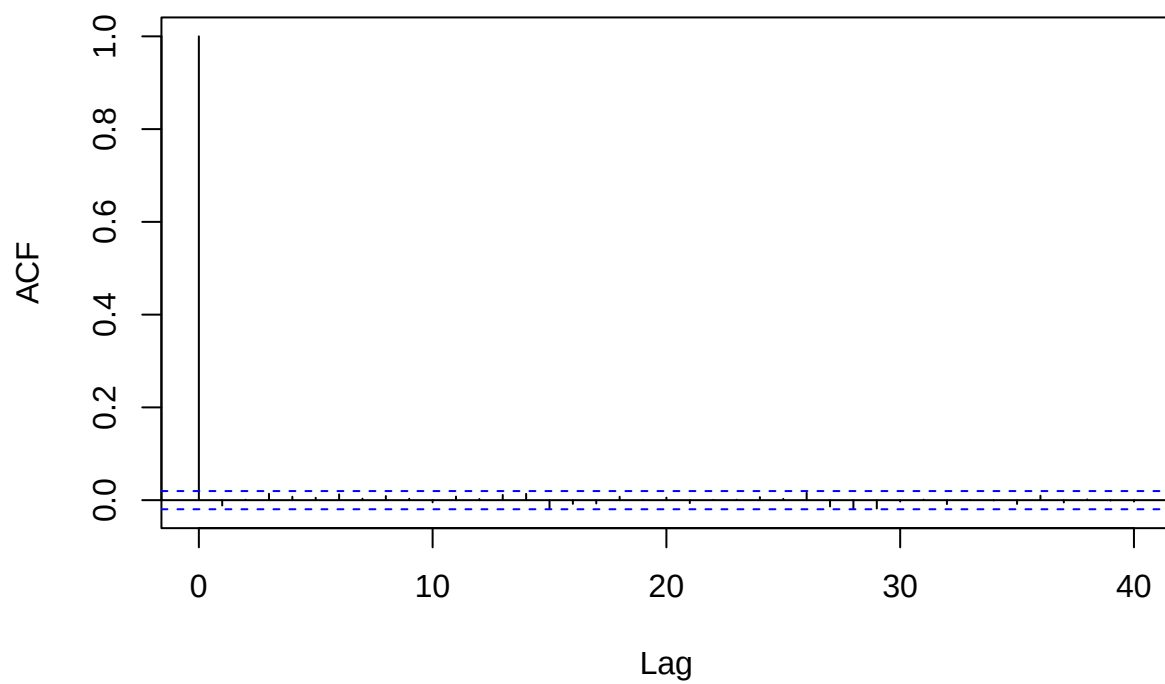
plot(x, type = "l", main = "Simulated ARCH(2)", ylab = "Value", xlab = "Time")
```

Simulated ARCH(2)



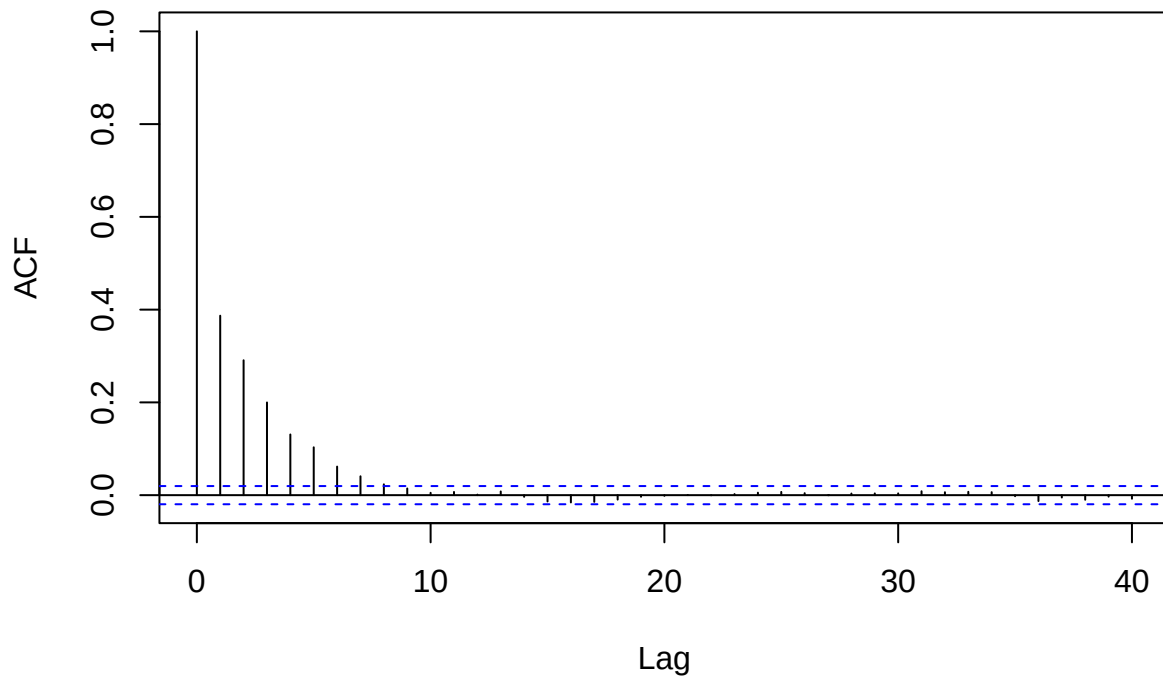
```
acf(x, main = "ACF of ARCH(2) series")
```

ACF of ARCH(2) series



```
acf(x2, main = "ACF of squared ARCH(2) series")
```

ACF of squared ARCH(2) series



The ARCH(2) model extends ARCH(1) by allowing the current variance to depend on the last two squared observations. As before, the raw series is nearly uncorrelated, while the squared series retains clear serial dependence.

1.4 Fitting GARCH models

Finally, we fit a GARCH model to the S&P500 returns.

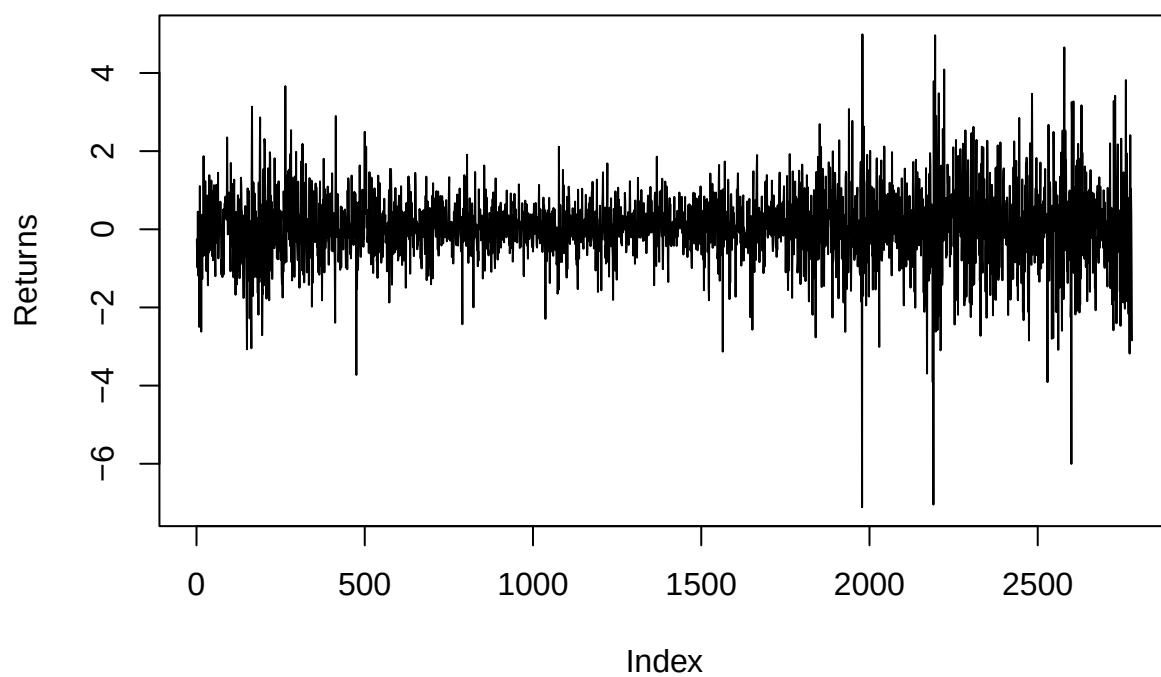
```
library(MASS)
library(tseries)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
data(SP500)
```

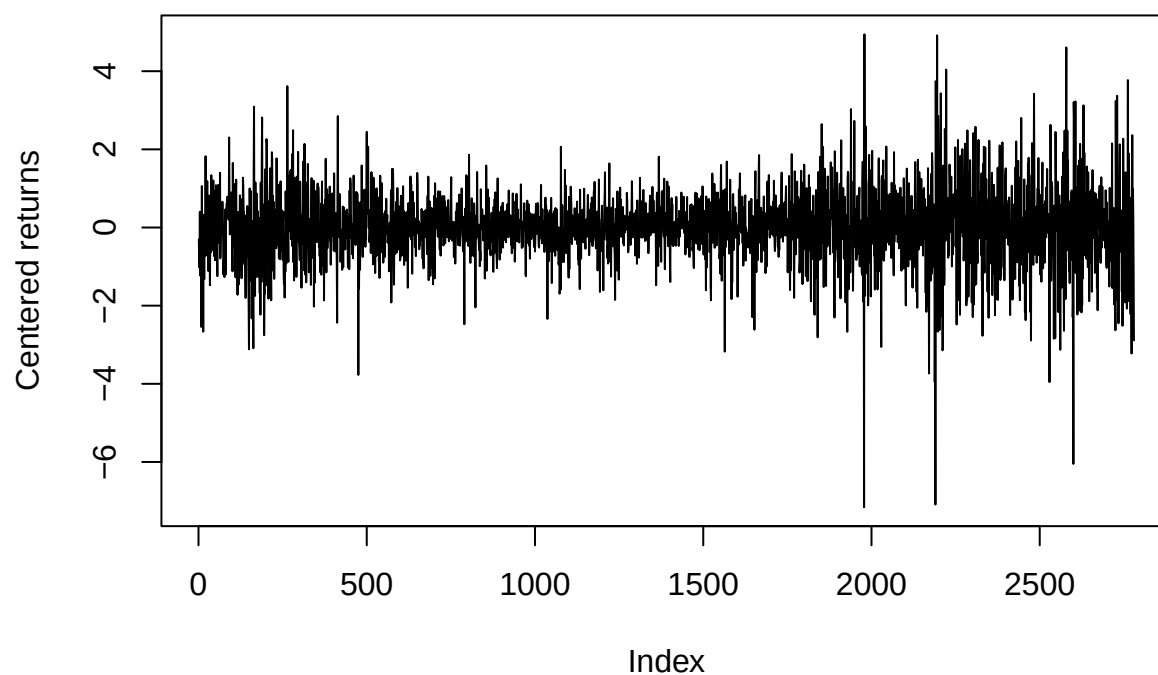
```
plot(SP500, type = "l", main = "S&P500 Daily Returns", ylab = "Returns")
```

S&P500 Daily Returns



```
x <- SP500 - mean(SP500)
plot(x, type = "l", main = "Centered S&P500 Returns", ylab = "Centered returns")
```

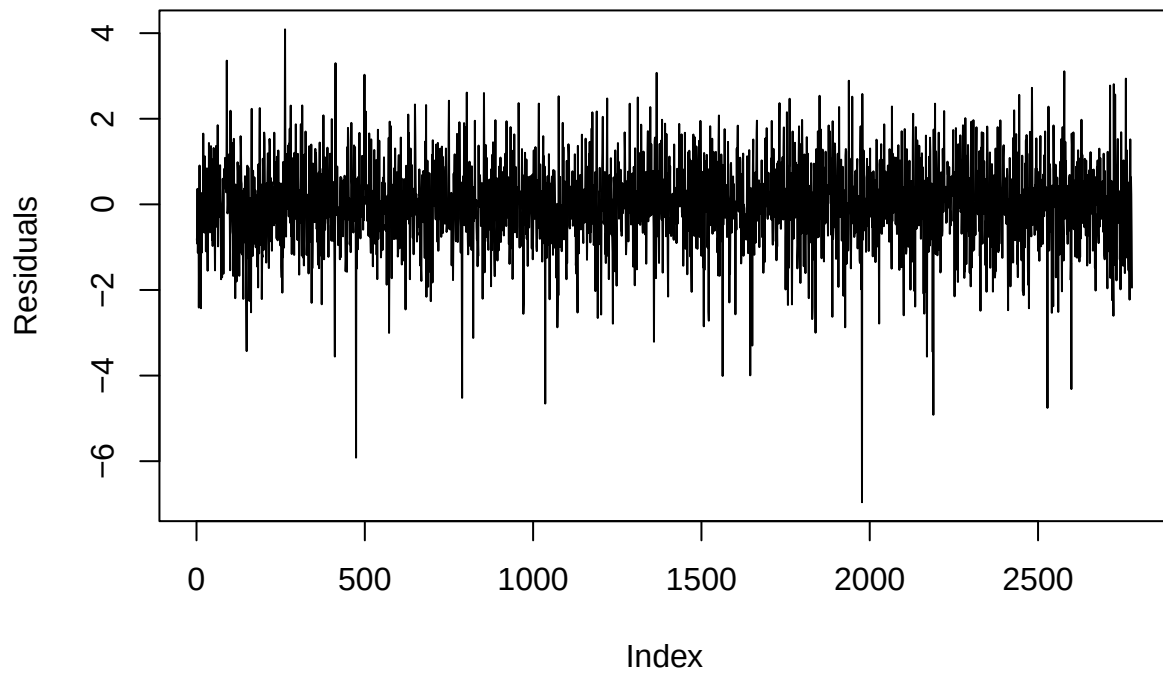
Centered S&P500 Returns



```
sp <- garch(x, trace = FALSE)
res <- sp$res[-1]

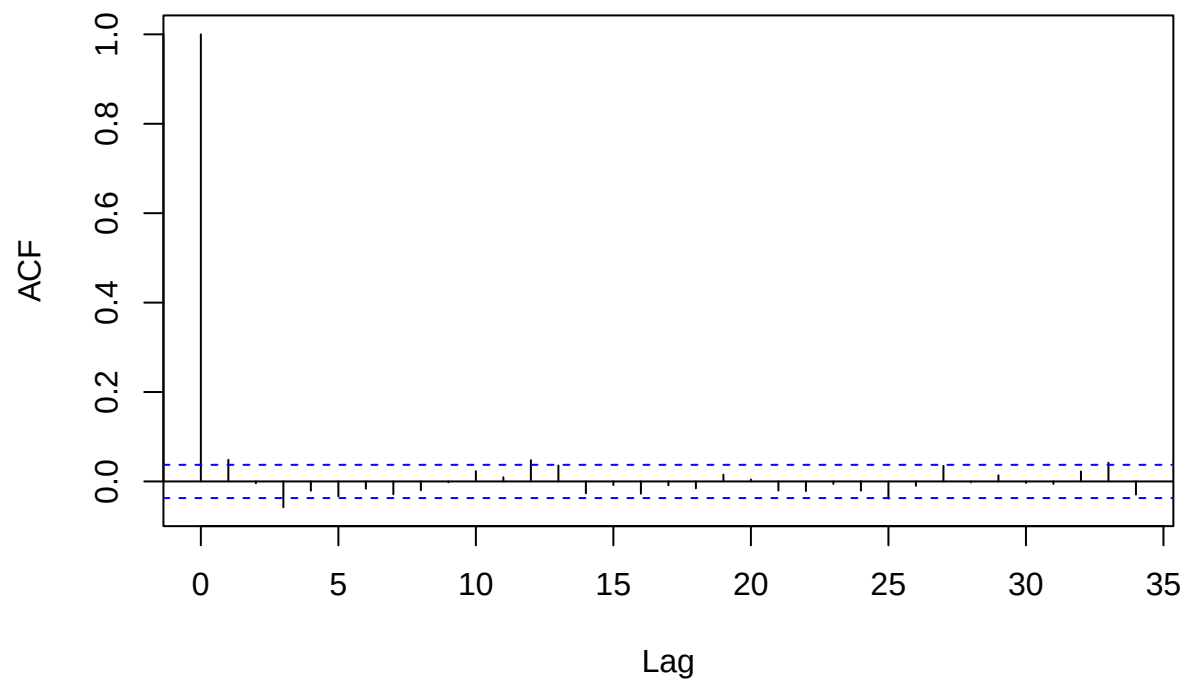
plot(res, type = "l", main = "GARCH residuals", ylab = "Residuals")
```

GARCH residuals



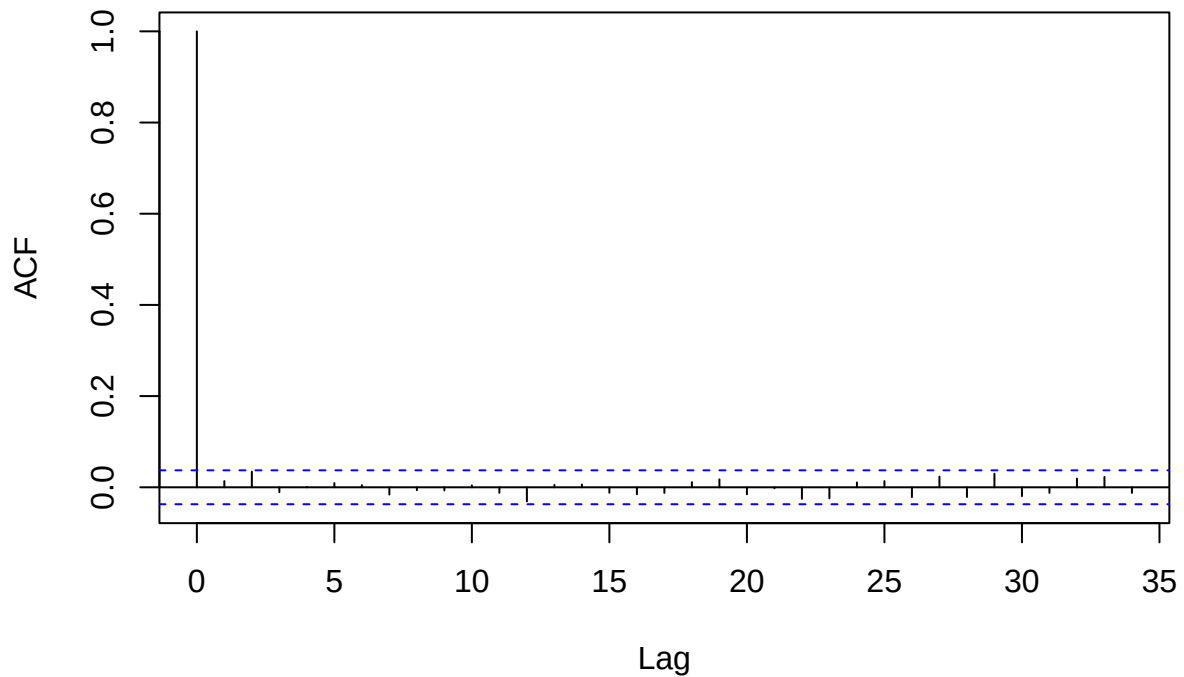
```
acf(res, main = "ACF of GARCH residuals")
```

ACF of GARCH residuals



```
acf(res^2, main = "ACF of squared GARCH residuals")
```

ACF of squared GARCH residuals



```
confint(sp)
```

```
##           2.5 %      97.5 %  
## a0 0.002455785 0.006734793  
## a1 0.043194690 0.060971683  
## b1 0.934964937 0.954021310
```

```
garch(x, order = c(1, 1), trace = FALSE)
```

```
##  
## Call:  
## garch(x = x, order = c(1, 1), trace = FALSE)  
##  
## Coefficient(s):  
##      a0      a1      b1  
## 0.004595 0.052083 0.944493
```

The fitted GARCH model should remove most of the volatility clustering from the centered returns. A good fit is suggested when the residuals and their squares no longer show strong autocorrelation, and the confidence intervals provide an idea of which parameters are clearly different from zero.